ENTERPRISE .NET REFERENCE SERIES

ASP.NET Core Testing Handbook

xUnit | Moq | Controller Testing | Deep Internals

■ Framework-Level Understanding	■■ Runtime Execution Internals
■ Reflection & Proxy Mechanisms	■■ Enterprise Architecture Patterns
■ xUnit Deep Internals	■ Moq Dynamic Proxy Internals
■ CI/CD & Pipeline Integration	■ Viva / Interview Preparation

10 Deep Sections • .NET 8/9 • Production Practices • May 2026

SECTION 1

Software Testing Fundamentals

1.1 What Testing Actually Means — The Real Definition

Most developers define testing as **checking if the code works**. That is dangerously shallow. Testing, architecturally, is the **systematic process of verifying that a software system behaves according to its specification under all anticipated and unanticipated conditions**, and — critically — producing **machine-readable, repeatable evidence** of that behavior.

The word *systematic* is key. Ad-hoc manual checks are not testing. Testing is a **formal discipline** with defined inputs, expected outputs, execution environments, and measurable outcomes. In the .NET ecosystem, a test is a **method that the runtime can discover, execute, and report on** without human intervention after the initial setup.

■ The Philosophical Core of Testing

Testing does not prove correctness. It proves the ABSENCE of specific defects under specific conditions. Edsger Dijkstra: 'Program testing can be used to show the presence of bugs, but never to show their absence.' Understanding this prevents over-confidence and shapes how you design test suites.

A test suite is a SPECIFICATION written in code. Every [Fact] method says: under these conditions, this component MUST behave this way. When a test passes, it's evidence. When it fails, it's a specification violation — either the code is wrong, or the specification changed.

1.2 Why Testing Exists — Architectural and Business Reasoning

Testing exists for six deep reasons that every senior engineer must internalize:

Reason 1: Cognitive Load Reduction

The human brain cannot simultaneously hold the entire state of a 100,000-line codebase. When you modify the BookingController, you cannot mentally verify that PassengerService, SeatAllocationService, and PaymentGatewayService still work correctly. A test suite does this verification automatically. **Tests are mechanical memory** — they remember every edge case you ever discovered.

Reason 2: Regression Prevention

Every bug fix and feature addition has **blast radius** — unintended effects on existing behavior. Without tests, regressions are discovered by users in production. With a comprehensive suite, regressions are caught in under 60 seconds on a developer's machine. The cost differential is enormous: a bug caught in development costs 1x; in QA costs 10x; in production costs 100x–1000x.

Reason 3: Architecture Enforcement

Code that is hard to test is architecturally defective. If you cannot instantiate BookingService without spinning up a database, a message queue, and an SMTP server, your class has violated the **Single Responsibility Principle** and has **tight coupling**. Testing pressure *forces* better architecture: dependency injection, interface segregation, and clean layering emerge naturally from the need to write testable code.

Reason 4: Living Documentation

A well-named test is a specification. `GetBooking_WithValidId_ReturnsOkWithBookingDto()` documents precisely what the system is supposed to do. Unlike Word documents, tests cannot lie — a passing test is always current documentation. New team members can understand system behavior by reading the test suite.

Reason 5: Refactoring Safety Net

Refactoring without tests is rewriting without a net. With a comprehensive suite, you can make aggressive architectural improvements — extract methods, change data structures, switch ORM providers — and know immediately if behavior has changed. This confidence accelerates development, not slows it.

Reason 6: CI/CD Enablement

Continuous deployment is impossible without automated testing. A pipeline that deploys on every commit requires machine-verifiable confidence. Tests are the gate that separates *change* from *deployment*. Without them, every deployment is a gamble.

1.3 Evolution of Software Testing — Historical Context

Understanding the history explains *why* modern testing tools are designed the way they are.

Era	Testing Approach	Problem It Solved
1950s–60s	Manual desk checking, hand tracing	No runtime environments existed
1970s	Black-box batch testing, test harnesses	Programs could now execute but were expensive
1980s	Structured testing, coverage metrics introduced	Large systems needed systematic approaches
1990s	Unit testing emerges (JUnit 1997), TDD concepts	Object-oriented code needed isolated validation
2000s	NUnit, MSTest, continuous integration	.NET ecosystem needed test automation
2010s	xUnit, Moq, BDD, mocking frameworks mature	DI-based architectures needed sophisticated mocking
2020s+	WebApplicationFactory, TestContainers, mutation testing	Microservices and cloud-native require integration confidence

1.4 The Cost of Bugs — Why This Matters Financially

The **IBM Systems Sciences Institute** and later studies confirmed the rule of exponential defect cost:

Phase Bug Detected	Relative Cost to Fix	Real-World Impact
During Development (unit test)	1x	Developer fixes in minutes
During Code Review	3x–5x	Discussion, rework cycle
During QA / Integration Testing	10x–15x	Test cycle restart, delay
In Staging / UAT	25x–50x	Business stakeholder involvement
In Production (low severity)	100x	Hotfix, deployment, rollback risk
In Production (data corruption)	1000x+	Legal, financial, reputation damage

1.5 Developer Mindset vs. Tester Mindset

This distinction is critical for designing good tests. Most developers have a **constructive mindset** — they think about how code *should* work. Testers have a **destructive mindset** — they think about every way the code can *fail*. Elite engineers can switch between both.

Aspect	Developer Mindset	Tester Mindset
Input focus	Valid, typical inputs	Invalid, boundary, unexpected inputs
Assumption	"It will work"	"How can this break?"
Test design	Happy path first	Failure paths first
State focus	Normal system state	Edge states, concurrent access, nulls
Goal	Feature completion	Defect discovery
Error handling	Writes it to handle errors	Actively tries to trigger those handlers

1.6 White-Box vs. Black-Box Testing — Internal Architecture Perspective

Black-Box Testing

The tester has **no knowledge of internal implementation**. Tests are written based purely on the specification: given input X, output must be Y. The test does not know how Y is produced. This mirrors how a client uses your API — they send an HTTP request and verify the response, not the internal logic.

When to use: API-level testing, acceptance testing, external integration testing. In controller testing, when you test endpoints through the HTTP pipeline, you are doing black-box testing.

White-Box Testing

The tester has **full knowledge of internal implementation**. Tests verify internal paths, branches, loops, and conditions. This requires knowing the source code. Unit testing is inherently white-box — you write tests knowing exactly which method you are testing and what its internal branches are.

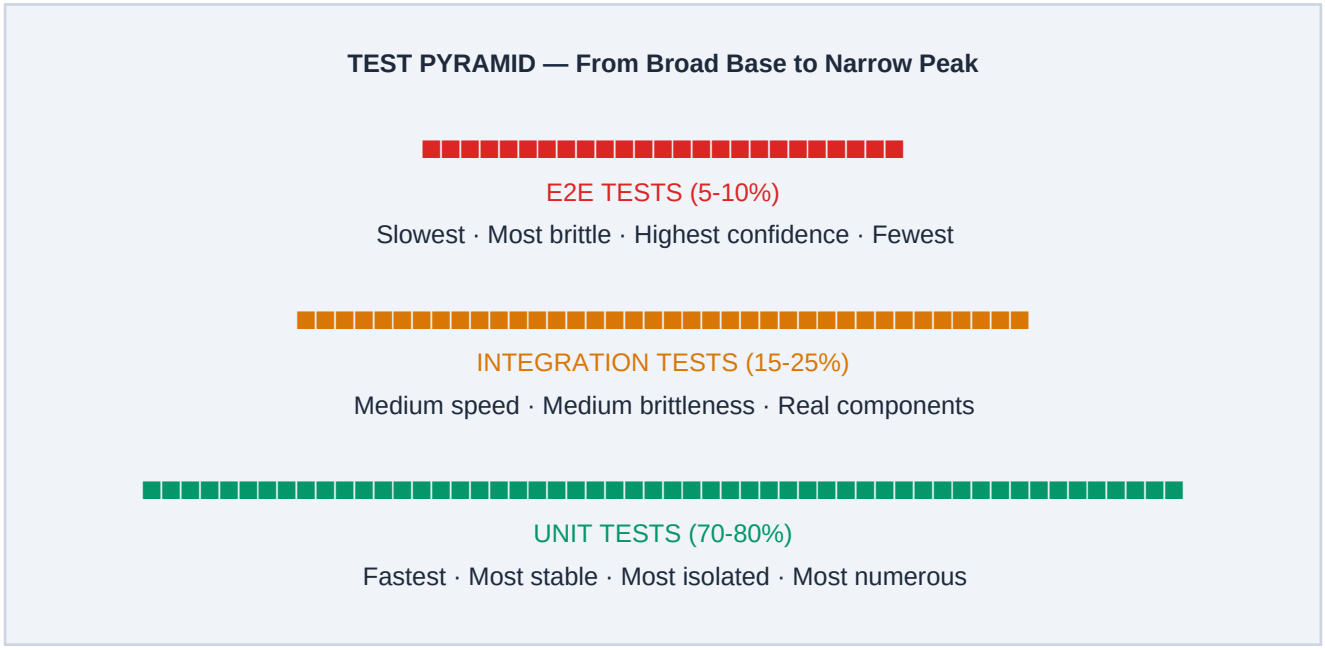
When to use: Unit testing individual methods, verifying specific code paths, branch coverage analysis. When you test that `GetBookingById` returns `NotFound` when the service throws `KeyNotFoundException`, you are doing white-box testing.

Gray-Box Testing

Partial knowledge of internals. Common in integration testing where you know the database schema but test through the API layer. Most real-world testing is gray-box.

1.7 The Test Pyramid — Architectural Guidance

Mike Cohn's Test Pyramid is not merely a ratio guideline — it is an **economic and architectural principle**. It answers: where should testing effort be concentrated to maximize confidence while minimizing maintenance cost and execution time?



Why this ratio? Each layer up the pyramid costs more: more setup, longer execution, more flakiness, harder debugging. A single Selenium E2E test might take 30 seconds and require a browser, database, and running server. A unit test takes 5ms and requires nothing. The pyramid says: *get maximum confidence from the cheapest tests*.

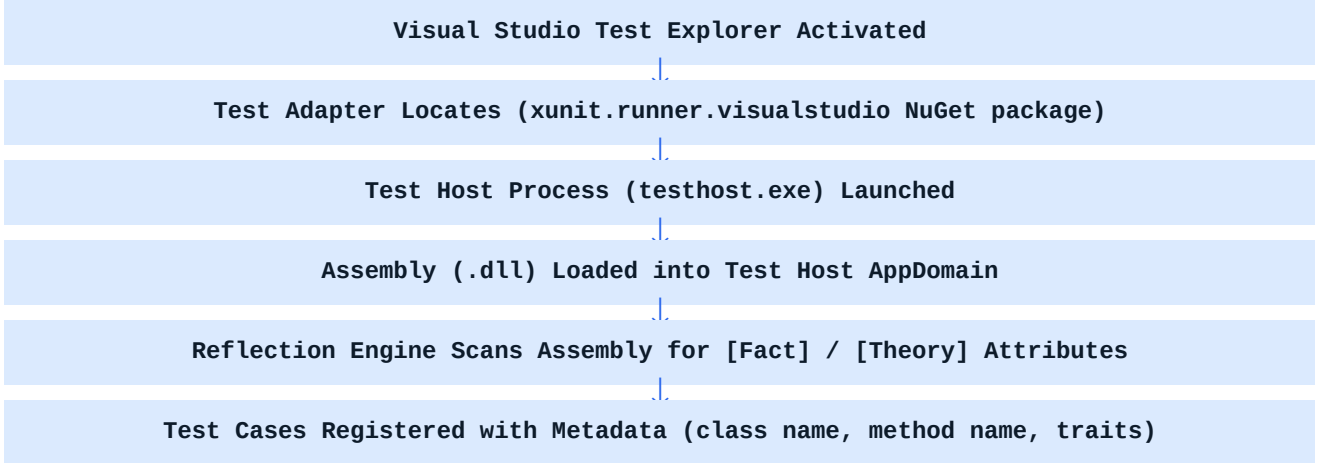
1.8 Shift-Left Testing — Philosophy

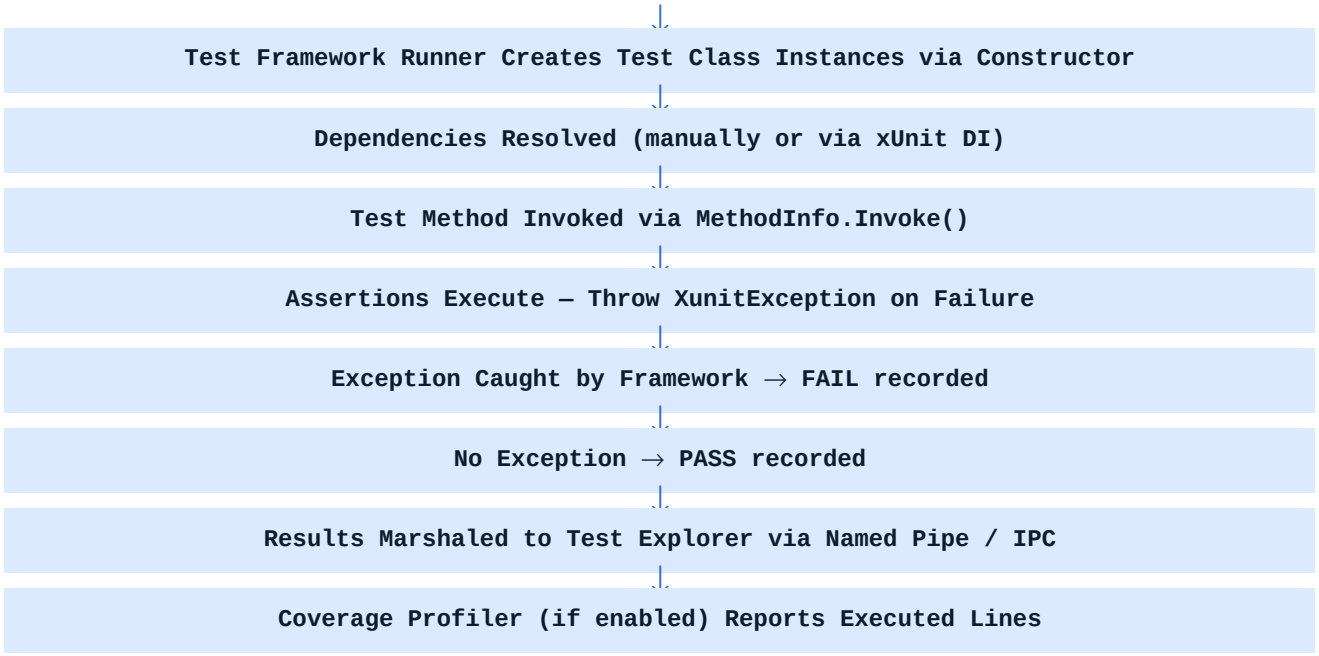
'Shift left' means moving testing activities **earlier** in the development lifecycle. Traditionally, testing was a phase that happened after development — a QA team received a build and tested it. Modern engineering rejects this model.

In a shift-left model, developers write tests *before or alongside* code (TDD), the CI pipeline runs tests on every commit, and QA engineers focus on exploratory and acceptance testing rather than regression checking. The cost curve of defect detection drops dramatically.

1.9 What Happens Internally During Test Execution — Runtime Mechanics

This is the hidden knowledge most developers lack. When you click 'Run Tests' in Visual Studio, a precise sequence of runtime operations occurs:





1.10 How Assertions Work Internally

This is widely misunderstood. An assertion is not magic — it is a **conditional throw**. Every assertion in xUnit ultimately calls `XunitException` (or a subclass) when the condition is not met. The test runner wraps each test method execution in a try-catch block. If any exception escapes — whether from an assertion or from the production code — the test is marked as FAILED.

```
// xUnit Assert.Equal internals (simplified conceptual representation)
public static void Equal<T>(T expected, T actual)
{
    // Uses EqualityComparer<T>.Default internally
    if (!EqualityComparer<T>.Default.Equals(expected, actual))
    {
        // Builds detailed failure message with type info
        string message = BuildFailureMessage(expected, actual);
        // Throws exception — test runner catches this
        throw new EqualException(expected, actual, message);
    }
    // If equal: method returns normally → test continues
}
// EqualException inherits from XunitException
// XunitException inherits from Exception
// Test runner: catch(Exception ex) { RecordFailure(ex.Message); }
```

Critical insight: If you write `Assert.Equal(5, result)` and result is 3, an `EqualException` is thrown. The test runner catches it, extracts the message ('Expected: 5, Actual: 3'), and marks the test FAILED. This is why assertions must be the *last* thing in a test — once one throws, subsequent assertions don't execute.

1.11 Viva / Interview Questions — Section 1

Q: What is the difference between verification and validation in testing?

A: Verification: Are we building the product RIGHT? (does it match specifications) — done via code review, unit tests, static analysis. Validation: Are we building the RIGHT product? (does it meet user needs) — done

via acceptance testing, UAT, demos. Unit testing is primarily verification; acceptance testing is validation.

Q: Why does testing improve architecture quality?

A: Because poorly-architected code is untestable. If a class has 10 dependencies instantiated via 'new' inside its constructor, you cannot test it in isolation. The pressure to write testable code forces loose coupling, interface segregation, and dependency injection — all of which are hallmarks of good architecture.

Q: How do assertions work internally in xUnit?

A: Every assertion is a conditional that throws XunitException (or a subclass) when the condition fails. The test runner wraps method execution in try-catch. An uncaught exception = test failure. Normal return = test pass. It's pure exception mechanics.

Q: What is the Test Pyramid and why does it matter?

A: The Test Pyramid is an economic principle: unit tests (70-80%) are cheapest, fastest, and most isolated. Integration tests (15-25%) are medium cost. E2E tests (5-10%) are most expensive, slowest, most brittle. The pyramid says to maximize confidence from the cheapest tests. Inverting the pyramid (mostly E2E) creates a slow, brittle, expensive test suite that developers stop trusting.

SECTION 2

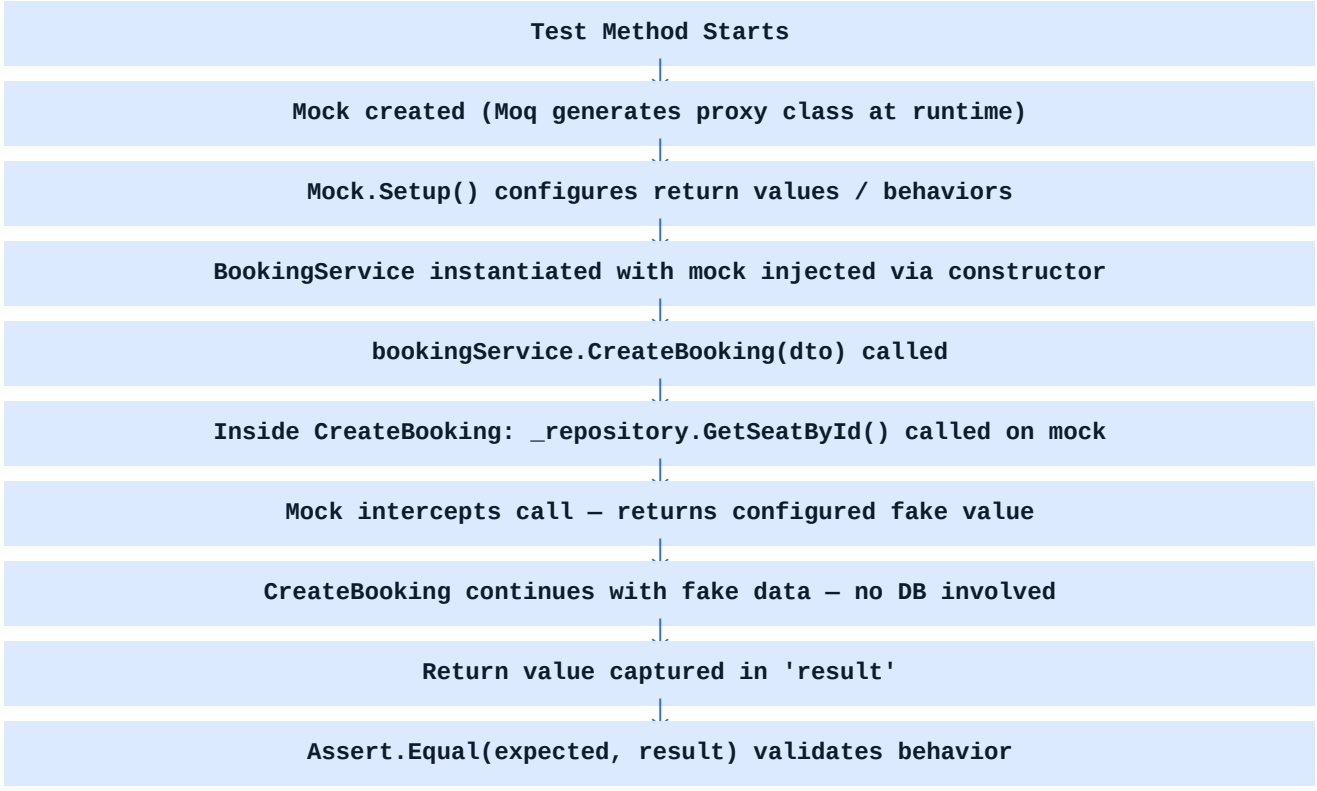
Types of Testing — Deep Internals and Architecture

2.1 Unit Testing — The Foundation

Definition: Testing a single *unit* of behavior in complete isolation from all external dependencies. A unit is typically a single method or a small cluster of methods that form one logical behavior.

What 'Isolation' Actually Means at Runtime

Isolation means that when BookingService.CreateBooking() is tested, it does not call the real IBookingRepository. Instead, a mock object — a dynamically generated fake class that implements IBookingRepository — is injected. The mock is configured to return predetermined values. This means the test's outcome depends ONLY on the logic inside CreateBooking(), not on the database, network, or file system.



What Unit Tests Should and Should NOT Verify

- **SHOULD test:** Business logic, conditional branches, calculations, transformations, input validation, exception throwing, service orchestration logic
- **SHOULD NOT test:** Framework code (ASP.NET routing), ORM queries (EF Core internals), database connectivity, third-party library behavior, configuration loading

Runtime Characteristics

- Execution time: 1ms–20ms per test (no I/O)

- No external process required — pure in-memory
- Fully deterministic — same input always produces same output
- Can run in parallel without conflicts (if properly isolated)
- Memory: each test creates fresh instances, GC cleans up between tests

2.2 Integration Testing — Where Real Components Meet

Integration testing verifies that multiple components work **correctly together**. Unlike unit testing, some or all real implementations are used. The goal is to catch interface mismatches, configuration errors, and behavioral incompatibilities that only manifest when components interact.

ASP.NET Core Integration Testing with WebApplicationFactory

In ASP.NET Core, `WebApplicationFactory<TStartup>` creates an in-memory test server that boots your actual application — loading middleware, controllers, DI registrations — but without binding to a real network port. HTTP requests are made via `HttpClient` that routes directly to the in-memory server.

```
// Integration test using WebApplicationFactory (.NET 8)
public class BookingApiIntegrationTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient  client;
    public BookingApiIntegrationTests(WebApplicationFactory<Program> factory)
    {
        // Override services for testing — replace real DB with in-memory
         client = factory.WithWebHostBuilder(builder =>
        {
            builder.ConfigureServices(services =>
            {
                // Remove real DbContext registration
                var descriptor = services.SingleOrDefault(
                    d => d.ServiceType == typeof(DbContextOptions<AppDbContext>));
                if (descriptor != null) services.Remove(descriptor);
                // Add in-memory database
                services.AddDbContext<AppDbContext>(options =>
                    options.UseInMemoryDatabase('TestDb'));
            });
        }).CreateClient();
    }
    [Fact]
    public async Task GetBooking_ReturnsOk_WhenBookingExists()
    {
        var response = await  client.GetAsync('/api/bookings/1');
        response.EnsureSuccessStatusCode();
        var content = await response.Content.ReadAsStringAsync();
        Assert.Contains('bookingId', content);
    }
}
```

Internal Flow of WebApplicationFactory

When `WebApplicationFactory<Program>` is instantiated:

1. It calls your app's `Program.cs` entry point through a special test bootstrapper, using `WebApplicationBuilder`
2. All middleware is registered exactly as in production
3. Service collection is built — your DI registrations apply
4. A `TestServer` (Microsoft.AspNetCore.TestHost) is created with an in-memory transport
5. `HttpClient` from `CreateClient()` sends requests directly to this TestServer via in-memory pipeline
6. Responses flow back through middleware, controller, and serialization exactly as in production

2.3 Functional Testing

Functional testing verifies that a feature works as a whole from the user's perspective, without concern for implementation details. It's black-box testing at the feature level. In APIs, this means testing an entire workflow: create booking → get booking → cancel booking → verify state.

Distinguished from integration testing by *scope*: integration testing focuses on component interactions; functional testing focuses on feature completeness against requirements.

2.4 End-to-End (E2E) Testing

E2E testing verifies the complete user journey through the entire system: browser → load balancer → API gateway → microservice → database → message queue → response. Tools: Selenium, Playwright, Cypress for UI; RestSharp or Postman/Newman for API E2E.

■■ E2E Test Challenges

E2E tests are valuable but dangerous if over-relied upon. They suffer from:

- FLAKINESS: Network timeouts, browser rendering timing, order dependency
- SLOWNESS: A suite of 500 E2E tests can take 2+ hours
- BRITTLENESS: UI changes break tests that have nothing to do with business logic
- DIAGNOSIS DIFFICULTY: When an E2E test fails, it could be any of 20 components

Enterprise rule: Keep E2E tests for critical USER JOURNEYS only, not feature coverage.

2.5 Regression Testing

Regression testing is the practice of re-running existing tests after every code change to verify that previously working functionality has not been broken. In modern CI/CD, regression testing is fully automated — the entire test suite runs on every commit.

Internal mechanism: The CI server checks out the commit, runs `dotnet test`, collects the results as TRX/XML files, and gates the merge if any test fails. Git branch protection rules enforce this gate. No human regression testing required.

2.6 Smoke Testing

A smoke test is a minimal, fast verification that the most critical paths of a system are operational after a deployment. The name comes from hardware testing: you power on a board and check if smoke appears (indicating catastrophic failure) before detailed testing.

In ASP.NET Core, smoke tests might hit `/health`, `/api/status`, and 2-3 critical endpoints. They run as the first stage in a deployment pipeline. If they fail, the deployment is rolled back immediately.

2.7 Performance, Load, and Stress Testing

Performance Testing

Measures response time, throughput, and resource utilization under defined load conditions. Tools: BenchmarkDotNet (micro-benchmarks), k6, JMeter, Azure Load Testing. Goal: verify that response time under normal load meets SLA requirements (e.g., 95th percentile < 200ms).

Load Testing

Gradually increases concurrent users/requests to find the system's capacity limit and behavior at scale. Identifies bottlenecks (database connection pool exhaustion, thread starvation, memory pressure) before they affect real users.

Stress Testing

Pushes the system beyond its expected capacity to observe failure behavior. Questions asked: Does it degrade gracefully? Does it recover after load drops? Does it corrupt data under extreme load?

2.8 Security Testing

Security testing verifies that the system resists attacks. Categories relevant to ASP.NET Core APIs:

- Authentication testing: verify JWT validation, expired token rejection, invalid signature handling
- Authorization testing: verify that role-based and policy-based access is enforced
- Input validation: SQL injection attempts, XSS payloads, buffer overflow attempts
- OWASP Top 10 verification: broken access control, security misconfiguration, injection flaws
- Penetration testing: automated (OWASP ZAP) and manual attempts to breach the system

In unit testing, security can be tested by mocking the `IAuthorizationService` and verifying that controllers correctly return 403 when authorization fails.

2.9 Viva Questions — Section 2

Q: What is the difference between unit testing and integration testing at the runtime level?

A: Unit tests: all external dependencies are mocked. Zero I/O. 1-20ms execution. Pure in-memory. Test one behavior in isolation. Integration tests: real components interact. May use real DB, real HTTP, real DI container. 100ms–10s execution. Test component interactions and interfaces.

Q: Why should unit tests never hit the real database?

A: Four reasons: (1) Speed — DB calls are 100-1000x slower than in-memory. (2) Isolation — a test's outcome should not depend on DB state left by other tests. (3) Reliability — DB unavailability would fail all tests unrelated to DB logic. (4) Clarity — if a test fails, you know it's a logic bug, not a data issue.

Q: What is WebApplicationFactory and how does it work internally?

A: It creates an in-memory ASP.NET Core application — boots Program.cs, registers all middleware, builds the DI container — but uses an in-memory transport (TestServer) instead of a real network port. HttpClient from CreateClient() sends requests directly to this in-memory server, going through the full middleware pipeline including routing, authentication, model binding, and serialization.

SECTION 3

xUnit Framework — Complete Internal Architecture

3.1 Why xUnit Exists — The Design Philosophy

xUnit.net was created by James Newkirk (co-creator of NUnit) and Brad Wilson specifically to fix fundamental design mistakes in NUnit and MSTest. Understanding *why* xUnit was redesigned explains every architectural decision in the framework.

Design Aspect	NUnit / MSTest Problem	xUnit Solution
Test class lifecycle	[TestFixture] class reused across tests → shared state	New class instance per test → guaranteed isolation
Setup/Teardown	[SetUp] / [TearDown] methods → hidden dependencies	Constructor / IDisposable → explicit, visible dependency
Parameterized tests	[TestCase] attribute → limited	[Theory] + [InlineData] / [MemberData] → extensible
Skip mechanism	[Ignore] attribute → permanent skip	Skip property on [Fact] → conditional skip
Extensibility	Plugin model was limited	xUnit built on IXunitTestCaseRunner interface → fully extensible
Parallelism	No built-in parallel execution	Parallel execution per collection, configurable
DI support	Not designed for DI	IClassFixture<T> and collection fixtures support DI patterns

3.2 xUnit Internal Architecture — Component Map

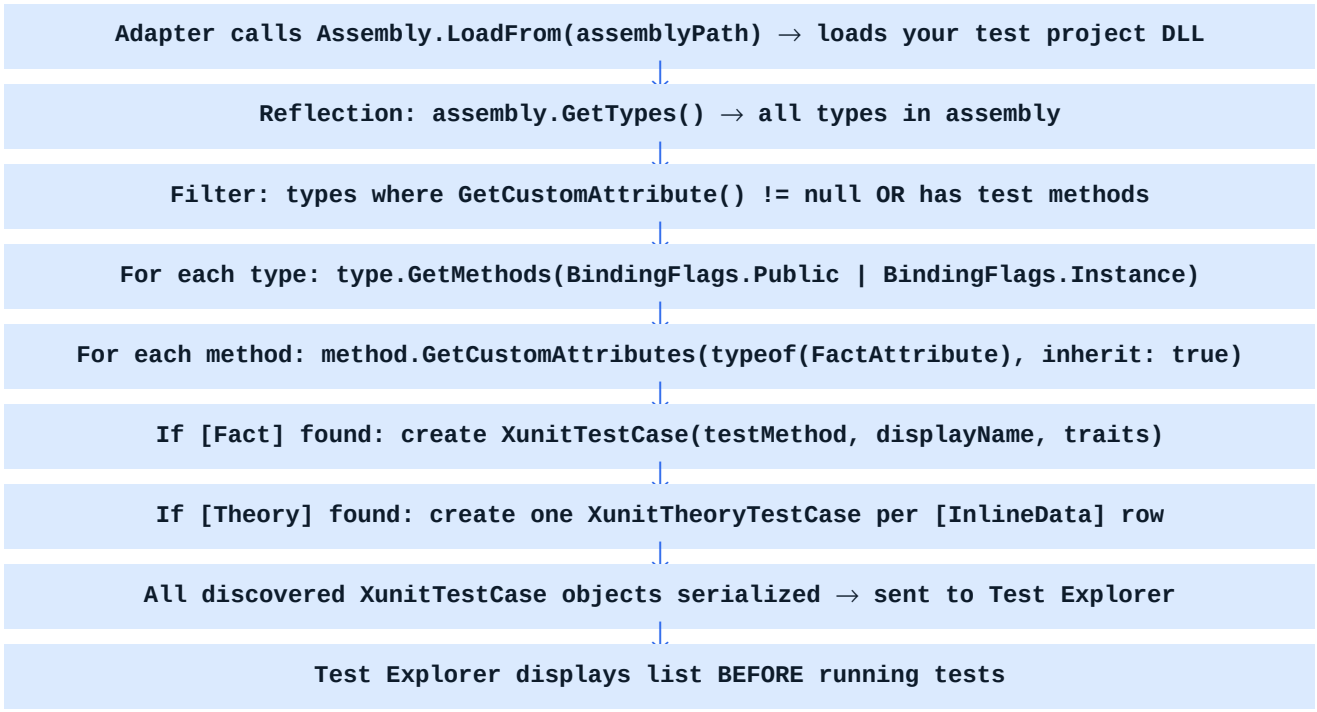
xUnit is composed of several assemblies, each with a specific responsibility:

- `xunit.core` — Core test framework: attributes ([Fact], [Theory]), assertions (Assert class), test case abstractions
- `xunit.execution.dotnet` — Execution engine: discovers, creates, and runs tests; manages parallelism and lifecycle
- `xunit.runner.visualstudio` — VSTest adapter: bridges xUnit to Visual Studio's Test Explorer via the VSTest protocol
- `xunit.runner.console` — Console runner for CI pipelines (used internally by `dotnet test`)
- `xunit.abstractions` — Interfaces and contracts: `ITestCase`, `ITestClass`, `ITestMethod`, `IXunitTestCase`

3.3 Test Discovery Internals — How xUnit Finds Tests

Test discovery is the process of identifying all test methods in an assembly before execution. This is entirely **reflection-based**. Here is the complete internal sequence:

```
dotnet test / vstest.console launches Test Host (testhost.exe)
                    ↓
xunit.runner.visualstudio (VSTest adapter) loaded
                    ↓
```



The [Fact] Attribute — What It Actually Does

The `[Fact]` attribute is a simple marker attribute — it contains no execution logic whatsoever. Its sole purpose is to act as metadata that the reflection scanner looks for. Here is its actual implementation concept:

```
// FactAttribute conceptual implementation
[AttributeUsage(AttributeTargets.Method, AllowMultiple = false)]
public class FactAttribute : Attribute
{
    // DisplayName shown in test explorer
    public virtual string DisplayName { get; set; }
    // Reason for skipping (non-null = test is skipped)
    public virtual string Skip { get; set; }
    // Timeout in milliseconds (0 = no timeout)
    public virtual int Timeout { get; set; }
    // IMPORTANT: No execution logic here.
    // The framework finds this attribute via reflection.
    // The runner calls the method via MethodInfo.Invoke().
}
// [Theory] inherits from [Fact] and adds IEnumerable<object[]> data concept
public class TheoryAttribute : FactAttribute { }
```

3.4 Test Execution Lifecycle — Step by Step

Once tests are discovered, execution begins. Understanding the object lifecycle is critical for understanding test isolation.

```
// xUnit test class instance lifecycle (what happens per test)
// Your test class:
public class BookingControllerTests
{
    private readonly BookingController _controller;
```

```
    private readonly Mock<IBookingService>  mockService;
    // CONSTRUCTOR runs for EVERY test method
    public BookingControllerTests()
    {
         mockService = new Mock<IBookingService>();  // Fresh mock every test
         controller = new BookingController( mockService.Object);  // Fresh controller
    }
    [Fact]
    public void Test1() { /* uses  controller and  mockService */ }
    [Fact]
    public void Test2() { /* gets DIFFERENT  controller and  mockService */ }
}
// xUnit execution sequence per test:
// 1. new BookingControllerTests() — fresh instance
// 2. Test1() invoked on that instance
// 3. Instance disposed if IDisposable
// 4. GC eligible — instance abandoned
// 5. new BookingControllerTests() — another fresh instance
// 6. Test2() invoked
```

Why this matters: Because a new class instance is created per test, instance fields are completely fresh. There is no way for Test1 to pollute Test2's state through instance variables. This is the xUnit isolation guarantee — and it's why the NUnit model of reusing the fixture class was considered a design flaw.

3.5 How the Test Runner Executes Tests — MethodInfo.Invoke()

After creating the test class instance, xUnit executes the test method using reflection's `MethodInfo.Invoke()`. This is not a direct method call — it goes through the reflection system, which allows xUnit to wrap it in try-catch and capture exceptions.

```
// Simplified test runner execution (conceptual)
// Inside xUnit's XunitTestRunner (simplified):
public async Task<RunSummary> RunTestAsync()
{
    var summary = new RunSummary();
    object testClassInstance = null;
    try
    {
        // 1. Create instance via reflection
        testClassInstance = Activator.CreateInstance(TestClass.Class.ToRuntimeType());
        // 2. Resolve constructor args (from IClassFixture<T> or test output)
        // (xUnit's DI is fixture-based, not full IoC container)
        // 3. Invoke the test method
        var result = TestMethod.ToRuntimeMethod().Invoke(testClassInstance, testMethodArgs)
;
        // 4. If async, await the Task
        if (result is Task task) await task;
        // 5. No exception = PASS
        summary.Total++;
```

```
            summary.Passed++;
        }
        catch (TargetInvocationException ex)
        {
            // Reflection wraps exceptions in TargetInvocationException
            var innerEx = ex.InnerException;
            if (innerEx is SkipException skip)
                summary.Skipped++;  // Skip
            else
            {
                summary.Failed++;   // Assertion failure or unexpected exception
                RecordFailure(innerEx);
            }
        }
        finally
        {
            // Dispose if IDisposable (runs async via IAsyncDisposable in .NET 8)
            if (testClassInstance is IDisposable d) d.Dispose();
        }
        return summary;
    }
```

3.6 Async Test Execution Internals

xUnit has first-class support for async tests. When a test method returns `Task` or `Task<T>`, xUnit awaits the task before declaring success or failure. This is critical — without framework support, async exceptions would be lost.

```
// Correct async test pattern
[Fact]
public async Task CreateBooking_WithValidRequest_ReturnsCreated()
{
    // Arrange
    var request = new CreateBookingRequest { PassengerId = 1, SeatId = 5 };
     mockService
        .Setup(s => s.CreateBookingAsync(request, It.IsAny<CancellationToken>()))
        .ReturnsAsync(new BookingDto { BookingId = 100 });
    // Act
    var result = await  controller.CreateBooking(request, CancellationToken.None);
    // Assert
    var created = Assert.IsType<CreatedAtActionResult>(result);
    Assert.Equal(100, ((BookingDto)created.Value).BookingId);
}
// COMMON MISTAKE: 'async void' tests are SILENT FAILURES
// [Fact]
// public async void Test() { ... }  // NEVER DO THIS
// If exception throws in async void, xUnit cannot catch it
// The test appears to PASS even when it fails
```

■ COMMON MISTAKE / ANTI-PATTERN

NEVER use 'async void' for test methods. xUnit awaits Task-returning tests. Async void methods launch fire-and-forget continuations that xUnit cannot observe. An assertion failure in async void will CRASH the test host process rather than marking the test as failed. Always use 'async Task'.

3.7 IClassFixture — Shared State Management

Sometimes you need expensive setup shared across all tests in a class (e.g., starting a Docker container, initializing a database seed). `IClassFixture<T>` provides this — the fixture is created once, shared across all tests in the class, then disposed.

```
// IClassFixture pattern — shared expensive resource
// Fixture: created ONCE per test class
public class DatabaseFixture : IDisposable
{
    public AppDbContext DbContext { get; private set; }
    public DatabaseFixture()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString()) // unique DB per fixture
            .Options;
        DbContext = new AppDbContext(options);
        DbContext.Database.EnsureCreated();
        SeedTestData(DbContext);
    }
    private void SeedTestData(AppDbContext ctx)
    {
        ctx.Bookings.Add(new Booking { BookingId = 1, PassengerId = 1 });
        ctx.SaveChanges();
    }
    public void Dispose() => DbContext?.Dispose();
}
// Test class: receives DatabaseFixture via constructor injection
public class BookingRepositoryTests : IClassFixture<DatabaseFixture>
{
    private readonly DatabaseFixture  fixture;
    public BookingRepositoryTests(DatabaseFixture fixture)
    {
         fixture = fixture; // shared across all tests in this class
    }
    [Fact]
    public void GetBooking ReturnsCorrectBooking()
    {
        var repo = new BookingRepository( fixture.DbContext);
        var booking = repo.GetById(1);
        Assert.NotNull(booking);
    }
}
// xUnit lifecycle with IClassFixture:
// 1. new DatabaseFixture() — once for the class
```

```
// 2. new BookingRepositoryTests(fixture) — per test
// 3. Test method executes
// 4. BookingRepositoryTests instance disposed
// 5. Repeat for each test
// 6. DatabaseFixture.Dispose() — once after all tests
```

3.8 Parallel Execution Internals

xUnit runs test collections in parallel by default. A **collection** is xUnit's unit of parallelism. By default, each test class is its own collection. Tests within a collection run sequentially; collections run in parallel.

```
// Controlling parallelism
// In xunit.runner.json (project root):
{
    'parallelizeAssembly': true,
    'parallelizeTestCollections': true,
    'maxParallelThreads': 4
}
// Force tests into same collection (sequential execution):
[Collection('Database Tests')]
public class BookingTests { }
[Collection('Database Tests')]
public class PassengerTests { }
// BookingTests and PassengerTests now run SEQUENTIALLY (same collection)
// Useful when sharing a real database that can't handle concurrent access
// Disable parallelism for a specific assembly:
[assembly: CollectionBehavior(DisableTestParallelization = true)]
```

3.9 Assertions Internals — The Full Assert Class

xUnit's `Assert` class contains ~50 assertion methods. Here are the most critical ones with their internal mechanisms:

Assert Method	Internal Mechanism	When to Use
Assert.Equal(expected, actual)	EqualityComparer<T>.Default.Equals() → throws EqualException if false	Value equality comparison
Assert.Same(expected, actual)	object.ReferenceEquals() → throws SameException if false	Reference identity (same object)
Assert.True(condition)	if (!condition) throw TrueException()	Boolean condition validation
Assert.Null(object)	if (object != null) throw NullException()	Null reference check
Assert.NotNull(object)	if (object == null) throw NotNullException()	Non-null reference check
Assert.IsType<T>(object)	if (object?.GetType() != typeof(T)) throw IsTypeException()	Exact type check
Assert.IsAssignableFrom<T>(obj)	if (!typeof(T).IsAssignableFrom(obj.GetType())) throw...	Type compatibility check
Assert.Throws<T>(action)	Invokes action; catches Exception; checks type; rethrows if wrong	Expected exception verification
Assert.ThrowsAsync<T>(func)	Awaits func(); catches exception; checks type	Async exception verification
Assert.Collection(list, ...)	Checks count matches inspectors count; runs each inspector on element	Sequential element inspection

Assert.Contains(item, collection)	Iterates collection with EqualityComparer; throws if not found	Collection membership

Assert.Throws — Internal Mechanism Deep Dive

```
// Assert.Throws internals (conceptual)
public static T Throws<T>(Action testCode) where T : Exception
{
    Exception caughtException = null;
    try
    {
        testCode(); // Execute the lambda that should throw
    }
    catch (Exception ex)
    {
        caughtException = ex;
    }
    // Case 1: No exception thrown — test fails
    if (caughtException == null)
        throw new ThrowsException(typeof(T), 'No exception was thrown.');
    // Case 2: Wrong exception type — test fails
    if (caughtException.GetType() != typeof(T))
        throw new ThrowsException(typeof(T), caughtException);
    // Case 3: Correct exception type — return it for further inspection
    return (T)caughtException;
}
// Usage:
var ex = Assert.Throws<ArgumentNullException>(
    () =>  controller.CreateBooking(null));
Assert.Equal('request', ex.ParamName); // Further inspection of exception
```

3.10 [Theory] and [InlineData] — Parameterized Tests

A `[Theory]` test runs the same test logic multiple times with different inputs. This is critically important for boundary testing — instead of duplicating test methods, you provide data rows.

```
// [Theory] with multiple data sources
// InlineData: hardcoded in attribute
[Theory]
[InlineData(1, true)]     // valid ID → should exist
[InlineData(0, false)]    // zero ID → invalid
[InlineData(-1, false)]   // negative → invalid
[InlineData(9999, false)] // non-existent → not found
public async Task GetBooking_ValidatesIdCorrectly(int id, bool expectSuccess)
{
     mockService.Setup(s => s.GetByIdAsync(1)).ReturnsAsync(new BookingDto());
     mockService.Setup(s => s.GetByIdAsync(It.Is<int>(i => i != 1)))
                .ReturnsAsync((BookingDto)null);
    var result = await  controller.GetBooking(id);
    if (expectSuccess)
```

```
        Assert.IsType<OkObjectResult>(result);
    else
        Assert.IsType<NotFoundResult>(result);
}
// MemberData: from static property (supports complex objects)
public static IEnumerable<object[]> InvalidRequests =>
    new List<object[]>
    {
        new object[] { null, 'Request cannot be null' },
        new object[] { new CreateBookingRequest { PassengerId = -1 }, 'Invalid passenger' }
,
        new object[] { new CreateBookingRequest { SeatId = 0 }, 'Invalid seat' },
    };
[Theory]
[MemberData(nameof(InvalidRequests))]
public async Task CreateBooking_WithInvalidRequest_ReturnsBadRequest(
    CreateBookingRequest request, string expectedError)
{
    var result = await  controller.CreateBooking(request);
    var badRequest = Assert.IsType<BadRequestObjectResult>(result);
    Assert.Contains(expectedError, badRequest.Value.ToString());
}
```

3.11 Viva Questions — xUnit Internals

Q: Why does xUnit create a new class instance per test method?

A: To guarantee test isolation. If the same instance were reused, state set in Test1's constructor or body could pollute Test2. NUnit reused instances (by default) and this caused hard-to-debug flaky tests. xUnit's per-test instantiation ensures every test starts with a clean, deterministic initial state.

Q: What is the difference between IClassFixture and a shared field in the constructor?

A: A shared field in the constructor creates a NEW instance per test (not shared). IClassFixture creates ONE instance for the entire test class and injects it into every test constructor. Use IClassFixture when setup is expensive (database, Docker container) and the resource can safely be shared (read-only access).

Q: How does xUnit discover test methods internally?

A: Via reflection: Assembly.LoadFrom() loads the DLL, assembly.GetTypes() gets all types, type.GetMethods() gets all public instance methods, method.GetCustomAttributes(typeof(FactAttribute)) checks for the marker. xUnit creates XunitTestCase objects for each discovered method and sends them to the test runner.

Q: Why does 'async void' in a test method cause silent failures?

A: xUnit awaits Task-returning methods. Async void methods return void — xUnit cannot await them. The async continuation runs fire-and-forget. If an assertion throws, the exception propagates on a thread pool thread with no test runner catch handler. The test process sees no exception and reports PASS, but the exception may crash the process or be silently swallowed.

SECTION 4

Moq Framework — Complete Internal Architecture and Mocking Internals

4.1 What Mocking Actually Means — The Philosophical Foundation

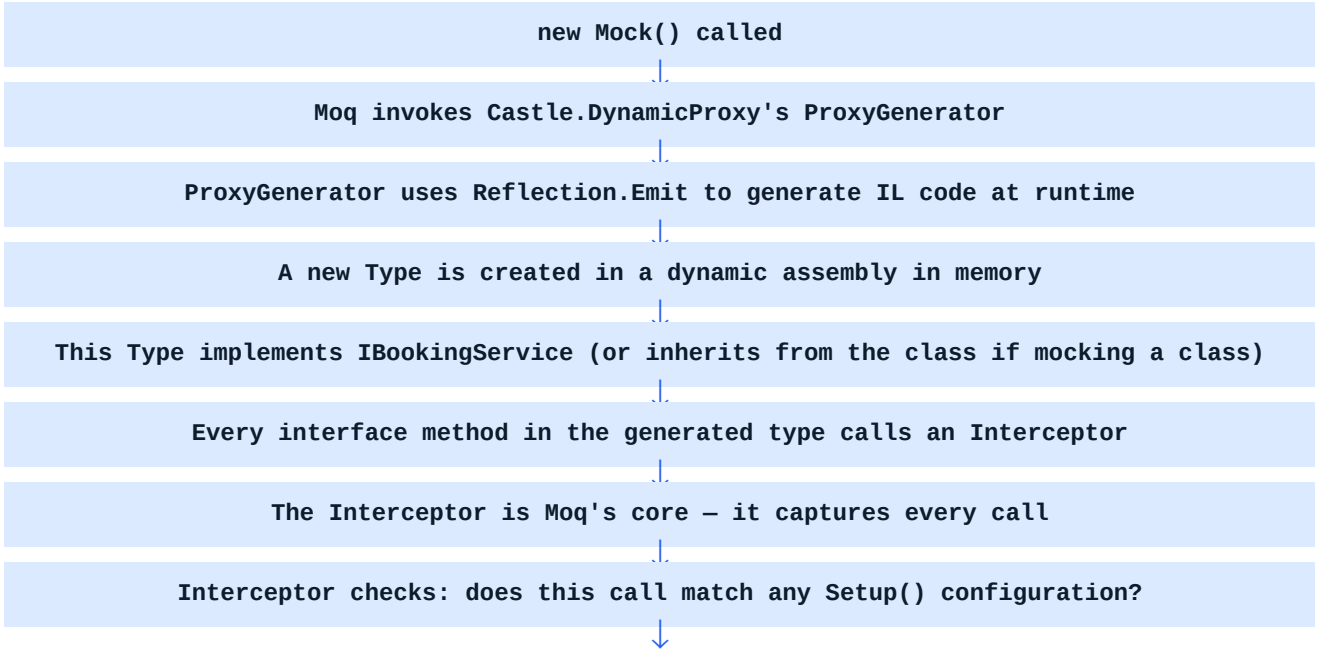
Mocking is often confused with 'faking' or 'stubbing'. These are related but distinct concepts rooted in Gerard Meszaros's foundational work on test doubles. Understanding the distinctions is critical for designing correct tests.

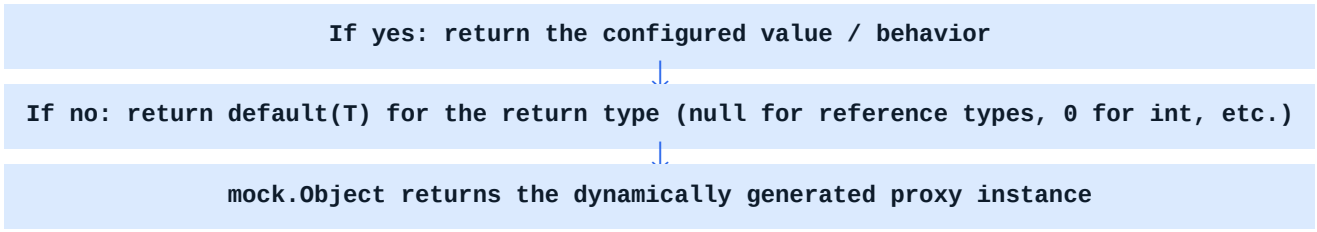
Test Double Type	What It Does	Has Assertions?	When to Use
Dummy	Passed but never used — just satisfies parameter requirements	No	When parameter is required but irrelevant to the test
Stub	Returns hardcoded values when called	No	no behavior verification When you need to control what a dependency returns
Fake	Real implementation but simplified (e.g., in-memory repo)	No	When behavior must be real but infrastructure is repla
Spy	Records calls made to it — can be inspected after	Manual inspection	When you need to observe interactions without strict p
Mock	Strict expectation verification — fails if unexpected calls made	Yes (automatic)	When verifying that specific interactions occurred exac

Critical insight: In Moq, `Mock<T>` can act as a stub (using `Setup()` to return values), a mock (using `Verify()` to check calls), or a spy (inspecting `mock.Invocations`). The type depends on how you use it, not which class you instantiate.

4.2 How Moq Creates Fake Objects — Dynamic Proxy Generation

This is the most misunderstood part of Moq. When you write `new Mock<IBookingService>()`, Moq does NOT create a simple object that returns nulls. It uses **Castle.DynamicProxy** (an embedded dependency) to **dynamically generate a real class at runtime** that implements the interface.





Why Interfaces Are Easy to Mock

When Moq mocks an interface, Castle.DynamicProxy generates a class that implements the interface. Every method in the interface becomes an overridable method in the generated class. The generated code for each method is essentially: call the interceptor, return whatever it says.

Why Non-Virtual Methods Are Hard to Mock

When mocking a concrete class (not an interface), Castle.DynamicProxy must *inherit* from the class and override its methods. But in C#, you can only override `virtual` methods. A non-virtual method cannot be intercepted — the proxy inherits it but cannot change its behavior. This is why mocking non-virtual methods on concrete classes fails.

```
// Why interface mocking works but concrete class non-virtual mocking fails
// This WORKS — interface, all methods are implicitly virtual in proxy
var mock = new Mock<IBookingService>();
mock.Setup(s => s.GetByIdAsync(1)).ReturnsAsync(new BookingDto());
// This WORKS — concrete class with virtual method
public class BookingService
{
    public virtual Task<BookingDto> GetByIdAsync(int id) { ... }
}
var mock = new Mock<BookingService>();
mock.Setup(s => s.GetByIdAsync(1)).ReturnsAsync(new BookingDto()); // WORKS
// This FAILS — non-virtual method on concrete class
public class BookingService
{
    public Task<BookingDto> GetByIdAsync(int id) { ... } // NOT virtual
}
var mock = new Mock<BookingService>();
// This compiles but the setup is IGNORED at runtime
// The real method executes instead of the interceptor
mock.Setup(s => s.GetByIdAsync(1)).ReturnsAsync(new BookingDto()); // SILENTLY FAILS
// This FAILS — sealed class
// Castle.DynamicProxy cannot inherit from a sealed class
var mock = new Mock<HttpClient>(); // throws MockException
// This FAILS — static methods
// Static methods belong to the type, not to instances
// No proxy can intercept static calls
// Solution: wrap static calls in a non-static service interface
```

4.3 Setup() Internals — Expression Trees

The `Setup()` method uses **C# Expression Trees** — not delegates, not strings, but a parsed representation of the lambda that can be analyzed at runtime. This is how Moq knows exactly which method you're configuring without calling it.

```
// How Setup() uses Expression Trees internally
// When you write:
mock.Setup(s => s.GetByIdAsync(1))
// The lambda 's => s.GetByIdAsync(1)' is NOT executed.
// C# compiles it as Expression<Func<IBookingService, Task<BookingDto>>>
// This is an Expression Tree — a data structure representing the code
// Moq receives this Expression Tree and analyzes it:
// - Body is a MethodCallExpression
// - MethodCallExpression.Method = MethodInfo for GetByIdAsync
// - MethodCallExpression.Arguments[0] = ConstantExpression with value 1
// Moq creates a MethodCall record:
// { Method: GetByIdAsync, Matcher: arg0 == 1, Behavior: ReturnsAsync(new BookingDto()) }
// When the proxy intercepts a call to GetByIdAsync(id):
// 1. Check all SetupCall records for this method
// 2. Does arg0 match the matcher (== 1)?
// 3. If yes: execute configured behavior (return Task<BookingDto>)
// 4. If no other setup matches: return default (null)
// It.IsAny<T>() creates a TypeMatcher instead of ConstantExpression
// It.Is<T>(pred) creates a PredicateMatcher
mock.Setup(s => s.GetByIdAsync(It.IsAny<int>()))  // matches ANY int argument
    .ReturnsAsync(new BookingDto());
```

4.4 Verify() Internals — Invocation Tracking

Every call to a mocked method is recorded by the interceptor. When you call `mock.Verify()`, Moq checks the invocation records against your specified expectations.

```
// Verify() and invocation tracking
// Every time mock.Object.GetByIdAsync(x) is called, Moq records:
// InvocationRecord { Method: GetByIdAsync, Arguments: [x], ReturnValue: ... }
// Verify checks invocation records against expectations:
// Exact call count:
mock.Verify(s => s.GetByIdAsync(1), Times.Once());
// Moq counts: how many InvocationRecords match (Method == GetByIdAsync AND arg0 == 1)?
// If count != 1: throw MockException('Expected once, actually called N times')
// At least once:
mock.Verify(s => s.DeleteBookingAsync(It.IsAny<int>()), Times.AtLeastOnce());
// Never called:
mock.Verify(s => s.SendNotificationAsync(It.IsAny<string>()), Times.Never());
// VerifyAll: checks ALL setups were called
mock.VerifyAll(); // fails if any Setup() was configured but not triggered
// VerifyNoOtherCalls: ensures no unexpected calls were made
mock.VerifyNoOtherCalls(); // fails if any call was made that wasn't verified
```

4.5 Setup Behavior Options — Complete Reference

```
// All Moq Setup behaviors
var mock = new Mock<IBookingService>();

// Returns a value
mock.Setup(s => s.GetByIdAsync(1)).ReturnsAsync(new BookingDto { BookingId = 1 });

// Returns null (for nullable reference types)
mock.Setup(s => s.GetByIdAsync(999)).ReturnsAsync((BookingDto)null);

// Throws an exception
mock.Setup(s => s.GetByIdAsync(-1))
    .ThrowsAsync(new ArgumentException('Invalid ID'));

// Callback — executes code when called (side effect)
int capturedId = 0;
mock.Setup(s => s.GetByIdAsync(It.IsAny<int>()))
    .Callback<int>(id => capturedId = id)  // capture the argument
    .ReturnsAsync(new BookingDto());

// Sequential returns — different value each call
mock.SetupSequence(s => s.GetStatusAsync())
    .ReturnsAsync('Pending')   // first call
    .ReturnsAsync('Confirmed') // second call
    .ReturnsAsync('Cancelled');// third call

// Return value based on input (dynamic)
mock.Setup(s => s.GetByIdAsync(It.IsAny<int>()))
    .ReturnsAsync((int id) => new BookingDto { BookingId = id });

// Verify property access
mock.SetupProperty(s => s.IsActive, true); // sets up get/set on the property
```

4.6 MockBehavior.Strict vs MockBehavior.Loose

`Mock<T>` has two behaviors that control what happens when an un-configured method is called:

```
// MockBehavior Strict vs Loose
// Loose (default): unconfigured calls return default values
var looseMock = new Mock<IBookingService>(MockBehavior.Loose);
// Calling GetByIdAsync(999) without Setup → returns null (no crash)

// Strict: unconfigured calls throw MockException immediately
var strictMock = new Mock<IBookingService>(MockBehavior.Strict);
// Calling GetByIdAsync(999) without Setup → throws MockException
// 'Invocation of GetByIdAsync was not expected.'

// When to use Strict:
// - When you want to catch accidental calls to dependencies
// - In pure unit tests where every dependency interaction should be explicit
// - Helps detect when refactoring adds unexpected dependency calls

// When to use Loose (default):
// - When the test focuses on one behavior and doesn't care about others
// - When mocking complex objects with many methods
// - Integration-style unit tests with many setup calls
```

4.7 Common Moq Mistakes — Anti-Patterns

■ COMMON MISTAKE / ANTI-PATTERN

ANTI-PATTERN 1: OVER-MOCKING — Mocking what you should be testing

If you mock the class under test, you're not testing it. Only mock DEPENDENCIES of the class under test

ANTI-PATTERN 2: SETUP NEVER TRIGGERED — Configuring a setup that the code never hits

This means you're testing a code path that doesn't exist. Use VerifyAll() to catch this

ANTI-PATTERN 3: VERIFY WITHOUT ASSERT — Only verifying interactions, not return values

A test that only checks 'was this method called' but not 'was the result correct' is incomplete

ANTI-PATTERN 4: It.IsAny() everywhere — Setups too broad to be meaningful

Setup(s => s.Process(It.IsAny())) means ANY call matches. You lose specificity

ANTI-PATTERN 5: Testing private methods via mocking — Mocking to bypass access modifiers

If you feel the need to test a private method, the design needs rethinking (extract to a service).

4.8 Viva Questions — Moq Internals

Q: How does Moq create a fake object at runtime?

A: Moq uses Castle.DynamicProxy to generate a real class at runtime using Reflection.Emit (IL generation). This class implements the mocked interface (or inherits from the mocked class for virtual methods). Every method in the generated class calls an Interceptor, which checks configured setups and returns the appropriate value or executes the configured behavior.

Q: Why can't you mock sealed classes or static methods with Moq?

A: Sealed classes: Castle.DynamicProxy works by creating a subclass. Sealed classes prohibit subclassing, so no proxy can be created. Static methods: they belong to the type, not to instances. Proxy objects only intercept instance method calls. Solutions: wrap in interfaces or use NSubstitute with NSubstitute.Analyzers, or use Microsoft Fakes (for static methods).

Q: What is the difference between Setup() and Verify() in Moq?

A: Setup(): configures what a mocked method should DO when called — return a value, throw, execute callback. Think of it as pre-configuration. Verify(): after execution, checks what ACTUALLY HAPPENED — was a method called, how many times, with what arguments. Setup shapes behavior; Verify validates interactions.

Q: What is MockBehavior.Strict and when should you use it?

A: Strict mode throws MockException immediately if any method is called that wasn't configured with Setup(). Use it when you want to explicitly account for every dependency interaction, catching accidental calls that shouldn't happen. The default (Loose) returns null/default for unconfigured calls, which is more forgiving but can hide unexpected interactions.

SECTION 5

ASP.NET Core Controller Testing — Deep Architecture

5.1 Why Controllers Are Tested — And What Makes a Good Controller Test

A controller in ASP.NET Core is responsible for four things: receiving HTTP input, delegating to services, transforming service results to HTTP responses, and handling errors. Controller tests verify **this orchestration logic** — not the service logic, not the database logic, not the HTTP parsing.

■ What Controller Tests Are NOT

Controller tests are NOT testing your business logic — that belongs in service tests.

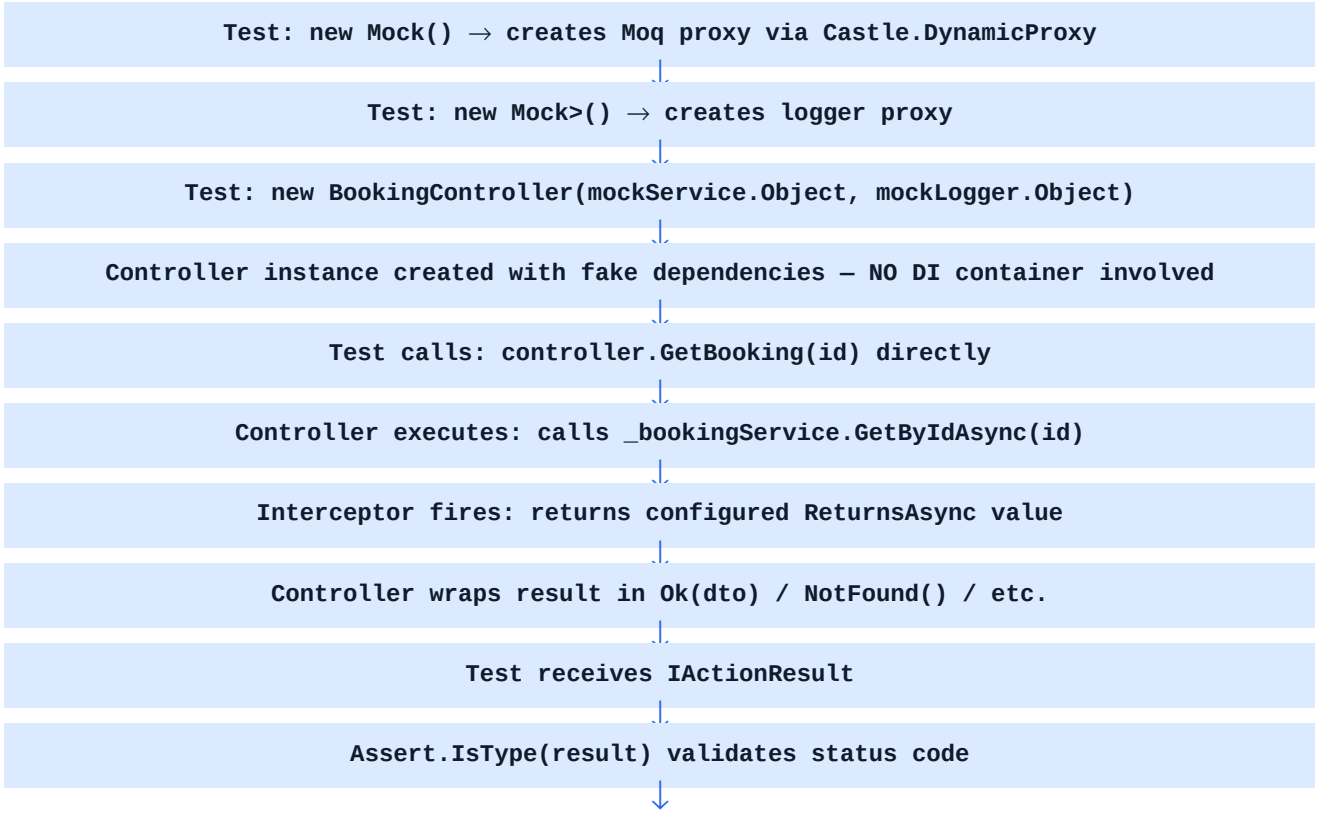
Controller tests are NOT testing your ORM queries — that belongs in repository tests.

Controller tests are NOT testing HTTP routing — that's ASP.NET Core framework responsibility.

Controller tests ARE testing: 1) correct action result type returned, 2) correct service method called, 3) correct HTTP status codes for each scenario, 4) error handling when service throws, 5) model validation behavior, 6) correct mapping between service response and API response.

5.2 Controller Activation During Tests — Internal DI Flow

In production, ASP.NET Core's DI container creates controller instances automatically — the framework calls `ActivatorUtilities.CreateInstance()` or the container directly resolves dependencies. In unit tests, **you manually create the controller**, injecting mock dependencies through the constructor.



Assert.Equal(expectedDto, okResult.Value) validates content

5.3 Complete Controller Test Architecture — BookingController

Below is a production-quality test class with full annotations explaining every design decision:

```
// BookingController — Production Code
[ApiController]
[Route('api/[controller]')]
public class BookingController : ControllerBase
{
    private readonly IBookingService  bookingService;
    private readonly ILogger<BookingController>  logger;
    public BookingController(IBookingService bookingService,
                             ILogger<BookingController> logger)
    {
        _bookingService = bookingService ?? throw new ArgumentNullException(nameof(bookingS
ervice));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }
    [HttpGet('{id:int}')]
    public async Task<IActionResult> GetBooking(int id)
    {
        if (id <= 0) return BadRequest('ID must be positive');
        var booking = await  bookingService.GetByIdAsync(id);
        if (booking == null) return NotFound();
        return Ok(booking);
    }
    [HttpPost]
    public async Task<IActionResult> CreateBooking([FromBody] CreateBookingRequest request)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        try
        {
            var booking = await _bookingService.CreateBookingAsync(request);
            return CreatedAtAction(nameof(GetBooking),
                new { id = booking.BookingId }, booking);
        }
        catch (SeatUnavailableException ex)
        {
            return Conflict(new { message = ex.Message });
        }
        catch (Exception ex)
        {
             logger.LogError(ex, 'Failed to create booking');
            return StatusCode(500, 'An unexpected error occurred');
        }
    }
    [HttpDelete('{id:int}')]
```

```
    public async Task<IActionResult> CancelBooking(int id)
    {
        var cancelled = await _bookingService.CancelBookingAsync(id);
        if (!cancelled) return NotFound();
        return NoContent();
    }
}
// BookingControllerTests — 4 Positive + 4 Negative (Full Production Quality)
public class BookingControllerTests
{
    // ■■ Dependencies (fresh per test, xUnit creates new instance per test)
    private readonly Mock<IBookingService> _mockService;
    private readonly Mock<ILogger<BookingController>> _mockLogger;
    private readonly BookingController _controller;
    public BookingControllerTests()
    {
        _mockService = new Mock<IBookingService>(MockBehavior.Strict);
        _mockLogger = new Mock<ILogger<BookingController>>();
        _controller = new BookingController(_mockService.Object, _mockLogger.Object);
    }
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // POSITIVE TEST CASES — GetBooking endpoint
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    [Fact]
    public async Task GetBooking_WithValidId_ReturnsOkWithBookingDto()
    {
        // Arrange: configure what the service returns for ID=1
        var expectedBooking = new BookingDto
        {
            BookingId = 1,
            PassengerId = 42,
            SeatNumber = 'A12',
            Status = 'Confirmed'
        };
        _mockService
            .Setup(s => s.GetByIdAsync(1))
            .ReturnsAsync(expectedBooking);
        // Act: call the controller action directly
        var result = await _controller.GetBooking(1);
        // Assert: verify type, status code, and content
        var okResult = Assert.IsType<OkObjectResult>(result);
        Assert.Equal(200, okResult.StatusCode);
        var returnedDto = Assert.IsType<BookingDto>(okResult.Value);
        Assert.Equal(1, returnedDto.BookingId);
        Assert.Equal('A12', returnedDto.SeatNumber);
        // Verify: service was called exactly once with correct ID
        _mockService.Verify(s => s.GetByIdAsync(1), Times.Once());
    }
```

```
[Fact]
public async Task CreateBooking_WithValidRequest_ReturnsCreated()
{
    // Arrange
    var request = new CreateBookingRequest { PassengerId = 5, SeatId = 10 };
    var createdBooking = new BookingDto { BookingId = 99, PassengerId = 5 };
    _mockService
        .Setup(s => s.CreateBookingAsync(request))
        .ReturnsAsync(createdBooking);
    // Act
    var result = await _controller.CreateBooking(request);
    // Assert
    var createdResult = Assert.IsType<CreatedAtActionResult>(result);
    Assert.Equal(201, createdResult.StatusCode);
    Assert.Equal(nameof(BookingController.GetBooking), createdResult.ActionName);
    var dto = Assert.IsType<BookingDto>(createdResult.Value);
    Assert.Equal(99, dto.BookingId);
}
[Fact]
public async Task CancelBooking_WithExistingId_ReturnsNoContent()
{
    // Arrange: service confirms cancellation
    _mockService
        .Setup(s => s.CancelBookingAsync(5))
        .ReturnsAsync(true);
    // Act
    var result = await _controller.CancelBooking(5);
    // Assert
    Assert.IsType<NoContentResult>(result);
    _mockService.Verify(s => s.CancelBookingAsync(5), Times.Once());
}
[Theory]
[InlineData(1)]
[InlineData(100)]
[InlineData(int.MaxValue)]
public async Task GetBooking_WithVariousValidIds_CallsServiceWithCorrectId(int id)
{
    // Arrange: any positive ID returns a booking
    _mockService
        .Setup(s => s.GetByIdAsync(id))
        .ReturnsAsync(new BookingDto { BookingId = id });
    // Act
    var result = await _controller.GetBooking(id);
    // Assert
    Assert.IsType<OkObjectResult>(result);
    _mockService.Verify(s => s.GetByIdAsync(id), Times.Once());
}
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
// NEGATIVE TEST CASES — Error paths and edge cases
// ═══════════════════════════════════════════════════════════════
[Fact]
public async Task GetBooking_WithNonExistentId_ReturnsNotFound()
{
    // Arrange: service returns null = not found
     mockService
        .Setup(s => s.GetByIdAsync(999))
        .ReturnsAsync((BookingDto)null);
    // Act
    var result = await  controller.GetBooking(999);
    // Assert: 404 returned, service was still called
    Assert.IsType<NotFoundResult>(result);
    _mockService.Verify(s => s.GetByIdAsync(999), Times.Once());
}
[Theory]
[InlineData(0)]
[InlineData(-1)]
[InlineData(-999)]
public async Task GetBooking_WithInvalidId_ReturnsBadRequest_WithoutCallingService(
    int invalidId)
{
    // Arrange: no setup — strict mock will throw if service called
    // Act
    var result = await  controller.GetBooking(invalidId);
    // Assert: controller rejects before calling service
    var badRequest = Assert.IsType<BadRequestObjectResult>(result);
    Assert.Contains('positive', badRequest.Value.ToString());
    // Critical: service should NOT have been called at all
     mockService.Verify(s => s.GetByIdAsync(It.IsAny<int>()), Times.Never());
}
[Fact]
public async Task CreateBooking_WhenSeatUnavailable_ReturnsConflict()
{
    // Arrange: service throws domain exception
    var request = new CreateBookingRequest { PassengerId = 1, SeatId = 5 };
    _mockService
        .Setup(s => s.CreateBookingAsync(request))
        .ThrowsAsync(new SeatUnavailableException('Seat 5 is already booked'));
    // Act
    var result = await  controller.CreateBooking(request);
    // Assert: domain exception maps to 409 Conflict
    var conflictResult = Assert.IsType<ConflictObjectResult>(result);
    Assert.Equal(409, conflictResult.StatusCode);
}
[Fact]
public async Task CreateBooking_WhenServiceThrowsUnexpectedly_Returns500()
{
```

```
        // Arrange: unexpected infrastructure exception
        var request = new CreateBookingRequest { PassengerId = 1, SeatId = 5 };
         mockService
            .Setup(s => s.CreateBookingAsync(request))
            .ThrowsAsync(new InvalidOperationException('Database connection lost'));
        // Act
        var result = await  controller.CreateBooking(request);
        // Assert: generic 500 returned (not internal details exposed)
        var statusResult = Assert.IsType<ObjectResult>(result);
        Assert.Equal(500, statusResult.StatusCode);
        // Also verify logger was called (infrastructure logging should happen)
         mockLogger.Verify(
            x => x.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.Is<It.IsAnyType>((o, t) => true),
                It.IsAny<Exception>(),
                It.IsAny<Func<It.IsAnyType, Exception, string>>()),
            Times.Once);
    }
}
```

5.4 Model State Testing

In controller unit tests, model validation via DataAnnotations does NOT run automatically. The ASP.NET Core model binder runs validation during HTTP request processing, which doesn't happen in unit tests. You must manually add errors to ModelState to simulate validation failures.

```
// Manual ModelState manipulation in tests
[Fact]
public async Task CreateBooking_WithInvalidModelState_ReturnsBadRequest()
{
    // Arrange: simulate validation failure
     controller.ModelState.AddModelError('PassengerId', 'PassengerId is required');
    var invalidRequest = new CreateBookingRequest(); // missing required fields
    // Act
    var result = await  controller.CreateBooking(invalidRequest);
    // Assert
    var badRequest = Assert.IsType<BadRequestObjectResult>(result);
    // Service should never be called when model state is invalid
     mockService.Verify(
        s => s.CreateBookingAsync(It.IsAny<CreateBookingRequest>()),
        Times.Never());
}
// WHY ModelState validation doesn't run in unit tests:
// Model binding (parsing request body, query string, route data) is done by
// the framework's model binder pipeline BEFORE the action executes.
// In unit tests, you call the action method directly — skipping the pipeline.
// Solution: use WebApplicationFactory for testing that validation works end-to-end.
```

5.5 Identifying Positive and Negative Test Cases — Enterprise Strategy

A systematic approach to test case design for each endpoint:

For GET /api/bookings/{id}:

- POSITIVE 1: Valid ID, booking exists → 200 OK with correct DTO
- POSITIVE 2: Valid ID at boundary (int.MaxValue) → 200 OK
- POSITIVE 3: Service returns booking with all fields populated → all fields in response
- POSITIVE 4: Multiple concurrent requests → all return correctly (parallelism test)
- NEGATIVE 1: ID = 0 → 400 Bad Request (before service call)
- NEGATIVE 2: ID = -1 → 400 Bad Request
- NEGATIVE 3: Valid ID, booking not found → 404 Not Found
- NEGATIVE 4: Service throws unexpected exception → 500 Internal Server Error

For POST /api/bookings:

- POSITIVE 1: Valid request → 201 Created with Location header
- POSITIVE 2: Minimum valid request → 201 Created
- POSITIVE 3: Request with optional fields → all fields in response
- POSITIVE 4: Booking created with correct ID in response
- NEGATIVE 1: Null request body → 400 Bad Request
- NEGATIVE 2: Invalid model state (missing required field) → 400 Bad Request
- NEGATIVE 3: Seat already booked (domain exception) → 409 Conflict
- NEGATIVE 4: Database failure → 500 Internal Server Error (no internal details exposed)

5.6 Viva Questions — Controller Testing

Q: Why don't DataAnnotation validations run in unit tests?

A: Because unit tests bypass the ASP.NET Core middleware pipeline. Model binding and validation run as part of the middleware pipeline, before the action method executes. In unit tests, you call the action method directly via a C# method call, skipping all middleware. To test validation end-to-end, use WebApplicationFactory integration tests. In unit tests, manually populate ModelState with AddModelError().

Q: Should controller tests verify that the service was called?

A: Yes, but selectively. Verify service calls when: (1) you're testing that a specific service method is invoked for a given scenario, (2) you want to confirm the service is NOT called in error paths (e.g., bad request). Don't verify every single call — tests should verify BEHAVIOR, not implementation. Over-verifying creates brittle tests that break on refactoring without behavior changes.

Q: What is the difference between Assert.IsType and Assert.IsAssignableFrom?

A: IsType: checks that the object's EXACT runtime type is T. A CreatedAtActionResult won't pass IsType even though it inherits from it. IsAssignableFrom: checks that the type is T OR inherits from T or implements T. In controller tests, use IsType when you want exact type (200 OK vs 201 Created both have StatusCode but are different types).

SECTION 6

Complete Test Execution Flow — What Happens Internally

6.1 The Complete Journey: Click 'Run Test' to Result

This section provides the most detailed explanation of what happens from the moment you click 'Run Tests' in Visual Studio or type 'dotnet test' in the terminal. This is the knowledge that distinguishes a senior engineer from a junior one.

Phase 1: Test Host Launch

Visual Studio's Test Explorer communicates with the test adapter layer (VSTest platform). When tests are requested:

```
// Process hierarchy when running tests
Visual Studio.exe (parent process)
  ■■■ vstest.console.exe OR dotnet test (orchestrator)
        ■■■ testhost.exe (per test assembly)
              ■■■ xunit.runner.visualstudio.dll (test adapter)
              ■■■ YourTestProject.dll (your test assembly)
              ■■■ All referenced assemblies loaded
Communication: Visual Studio ↔ vstest via Named Pipes (IPC)
Communication: vstest ↔ testhost via Named Pipes
Protocol: VSTest Protocol (binary serialization over named pipe)
```

Phase 2: Assembly Loading

The test host process loads your test assembly using `Assembly.LoadFrom(assemblyPath)`. This is a full CLR load — all referenced assemblies are resolved and loaded transitively. The CLR's fusion (assembly resolution) algorithm applies: GAC → application base directory → private paths.

Phase 3: Test Discovery via Reflection

This is the most computationally expensive phase. xUnit's discovery engine runs:

```
// Test discovery detailed sequence
// 1. Get all types in assembly
Type[] allTypes = assembly.GetTypes();
// 2. Filter to non-abstract, non-generic public types
var candidateTypes = allTypes.Where(t =>
    !t.IsAbstract && !t.IsGenericTypeDefinition &&
    t.IsClass && t.IsPublic);
// 3. For each type, check for test methods
foreach (var type in candidateTypes)
{
    // Get all public instance methods
```

```
    var methods = type.GetMethods(
        BindingFlags.Public | BindingFlags.Instance | BindingFlags.FlattenHierarchy);
    foreach (var method in methods)
    {
        // Check for [Fact] attribute (or subclass like [Theory])
        var factAttr = method.GetCustomAttribute<FactAttribute>(inherit: true);
        if (factAttr == null) continue;
        // Extract metadata
        string displayName = factAttr.DisplayName ?? $"{type.Name}.{method.Name}";
        string skipReason = factAttr.Skip;
        var traits = method.GetCustomAttributes<TraitAttribute>();
        // Create test case record
        var testCase = new XunitTestCase(
            testMethod: new ReflectionMethodInfo(method),
            displayName: displayName,
            skipReason: skipReason,
            traits: traits.ToDictionary(t => t.Name, t => t.Value)
        );
        testCases.Add(testCase);
    }
}
// [Theory] methods create MULTIPLE test cases — one per data row
// InlineDataAttribute provides object[] rows
// Each row becomes a separate XunitTestCase with arguments
```

Phase 4: Test Object Creation

For each test case, xUnit creates a fresh instance of the test class using `Activator.CreateInstance()` (or `ActivatorUtilities.CreateInstance()` for DI-aware fixtures):

```
// Test object instantiation internals
// Simple case — no IClassFixture
var testInstance = Activator.CreateInstance(testClass.ToRuntimeType());
// With IClassFixture<T> — fixture injected via constructor
// xUnit has already created (or cached) the fixture:
var fixture = GetOrCreateFixture(fixtureType);
var testInstance = Activator.CreateInstance(
    testClass.ToRuntimeType(),
    new object[] { fixture }  // injected into constructor
);
// With multiple constructor parameters:
// xUnit resolves: ITestOutputHelper (built-in), IClassFixture<T>, ICollectionFixture<T>
// Your constructor signals what you need:
public class MyTests : IClassFixture<DatabaseFixture>
{
    public MyTests(
        DatabaseFixture fixture,         // resolved from IClassFixture
        ITestOutputHelper output)        // resolved from xUnit's built-in
    { }
}
```

Phase 5: Test Method Execution

The test method is invoked through reflection. For async methods, the returned Task is awaited:

```
// Method execution with exception capture
object returnValue;
try
{
    // Invoke via reflection — TargetInvocationException wraps any thrown exception
    returnValue = testMethod.Invoke(testInstance, methodArguments);
}
catch (TargetInvocationException ex)
{
    // Unwrap TargetInvocationException to get the real exception
    ExceptionDispatchInfo.Capture(ex.InnerException).Throw();
}
// Handle async return
if (returnValue is Task task)
{
    // xUnit awaits the task — this is how async tests work
    task.GetAwaiter().GetResult(); // simplified; actually uses ConfigureAwait patterns
}
// After method completes normally: record PASS
// After exception: record FAIL with exception message and stack trace
```

Phase 6: Assertion Validation

Assertions run synchronously as part of the test method body. Each `Assert.*` call either returns normally (pass) or throws `XunitException` (fail). The exception propagates up to the runner's catch block, which records the failure message.

Phase 7: Result Reporting

Test results are serialized and sent back to the test orchestrator via Named Pipe IPC. Each result contains: test name, pass/fail/skip, duration, failure message, stack trace. Visual Studio's Test Explorer receives these and updates the UI. For CI pipelines, results are written to TRX (Visual Studio test results XML) or JUnit XML format for tools like Azure DevOps and GitHub Actions.

6.2 Code Coverage Tools — How They Work Internally

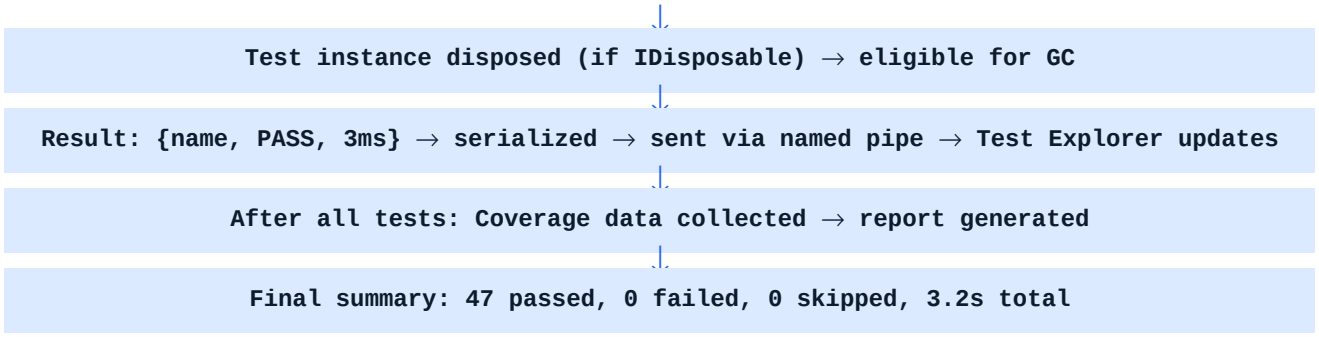
Coverage tools (Coverlet, dotCover) work via **instrumentation**. Before tests run, every line of production code is modified to include counter increments. When the instrumented code executes, counters record which lines were hit. After tests complete, the counters are read to produce a coverage report.

Coverlet's Two Instrumentation Modes

- **MSBuild integration:** Modifies the assembly after compilation but before tests, using `Mono.Cecil` to rewrite IL code
- **DataCollector mode:** Uses the CLR's profiling API (`ICorProfilerCallback`) to intercept every instruction execution — zero IL modification required

```
// Running tests with coverage (CLI)
# Coverlet via MSBuild integration
dotnet test --collect:'XPlat Code Coverage'
# Generate detailed HTML report with ReportGenerator
reportgenerator -reports:coverage.xml -targetdir:coveragereport -reporttypes:Html
# Coverage thresholds (fail if below 80%)
dotnet test /p:CollectCoverage=true /p:CoverletOutput=coverage/ \
            /p:CoverageFailUnder=80
# In CI/CD pipeline (Azure DevOps):
# - Collect coverage
# - Publish as artifact
# - Display in build results
# - Gate on coverage threshold
```

6.3 Complete Execution Flow Diagram



6.4 Viva Questions — Test Execution Internals

Q: What happens exactly between 'dotnet test' and the first test executing?

A: dotnet test launches vstest.console. vstest spawns testhost.exe for each test assembly. testhost loads the assembly via Assembly.LoadFrom(). xunit.runner.visualstudio (VSTest adapter) activates. It uses reflection to scan all types and methods for [Fact]/[Theory] attributes. Test cases are registered with metadata. The execution engine creates test class instances via Activator.CreateInstance() and invokes methods via MethodInfo.Invoke().

Q: How does code coverage work technically?

A: Coverlet instruments the assembly by injecting counter increment calls at the IL level, using Mono.Cecil to rewrite the compiled bytecode before tests run. When tests execute instrumented code, counters record which lines were hit. After tests complete, counters are read and compared against the source map to produce line/branch/method coverage percentages. The CLR profiling API mode achieves the same without IL modification, using runtime hooks.

Q: How does Visual Studio know a test passed or failed?

A: testhost.exe communicates with vstest.console via Named Pipes (IPC). Test results are serialized using the VSTest protocol and streamed: each result contains test name, pass/fail status, duration, failure message, and stack trace. vstest forwards these to Visual Studio, which updates the Test Explorer UI in real-time.

SECTION 7

Testing Best Practices — Production-Quality Testing

7.1 Arrange-Act-Assert (AAA) Pattern — The Structural Foundation

The AAA pattern is not a stylistic preference — it is an **architectural pattern for test clarity**. Every test has exactly three phases, clearly separated. This structure makes tests readable, debuggable, and maintainable.

```
// Perfect AAA structure with annotations
[Fact]
public async Task GetBooking_WithValidId_ReturnsCorrectBooking()
{
    // ■■■ ARRANGE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Setup: configure ALL preconditions for this test
    // - What data exists?
    // - What do mocks return?
    // - What is the system state?
    var bookingId = 42;
    var expectedBooking = new BookingDto
    {
        BookingId = bookingId,
        PassengerId = 1,
        SeatNumber = 'B5'
    };
    _mockService
        .Setup(s => s.GetByIdAsync(bookingId))
        .ReturnsAsync(expectedBooking);
    // ■■■ ACT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Execute: call the SINGLE operation being tested
    // One line is ideal. If you need multiple lines, consider if
    // you're testing multiple behaviors (split into multiple tests)
    var result = await _controller.GetBooking(bookingId);
    // ■■■ ASSERT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Verify: check that the outcome matches expectations
    // Multiple asserts are fine when they all verify the SAME behavior
    var okResult = Assert.IsType<OkObjectResult>(result);
    var returnedDto = Assert.IsType<BookingDto>(okResult.Value);
    Assert.Equal(bookingId, returnedDto.BookingId);
    Assert.Equal('B5', returnedDto.SeatNumber);
    // Interaction assertion (optional, when relevant)
    _mockService.Verify(s => s.GetByIdAsync(bookingId), Times.Once());
}
```

7.2 Test Naming Conventions — The Documentation System

Test names are the primary documentation of your system's behavior. A well-named test tells anyone reading it what the system does, under what condition, with what result.

The Gold Standard: MethodName_Condition_ExpectedResult

- `GetBooking_WithValidId_ReturnsOkWithBookingDto`
- `CreateBooking_WhenSeatUnavailable_ReturnsConflict`
- `CancelBooking_WithNonExistentId_ReturnsNotFound`
- `GetAllBookings_WithNoBookings_ReturnsEmptyList`
- `CreatePassenger_WithDuplicateEmail_ReturnsBadRequest`

Pattern Component	Purpose	Example
MethodName	WHAT is being tested	GetBooking, CreateBooking, CancelSeat
Condition (When/With)	CONTEXT of the test	WithValidId, WhenServiceThrows, WithNullRequest
ExpectedResult	WHAT should happen	ReturnsOk, ThrowsException, CallsServiceOnce

7.3 Test Independence — Critical Architectural Requirement

Every test MUST be completely independent of every other test. This means:

- No shared mutable state between tests (in-memory databases, static variables, files)
- No test order dependency — tests must produce the same result regardless of execution order
- No test must set up data for another test — each test is self-contained
- xUnit's per-test instantiation helps, but shared fixtures (IClassFixture) can break this if mutated

■ COMMON MISTAKE / ANTI-PATTERN

```
ANTI-PATTERN: Static state shared between tests
static List sharedList = new(); // NEVER in test class
[Fact] Test1() { sharedList.Add(new Booking()); }
[Fact] Test2() { Assert.Single(sharedList); } // FLAKY — depends on Test1 running first
ANTI-PATTERN: IClassFixture with mutable shared state
If DatabaseFixture contains shared AppDbContext and Test1 adds records that Test2 reads
test order changes the result. Solution: use unique GUIDs for in-memory DB names per test,
or clean up in each test via DbContext.Database.EnsureDeleted() / EnsureCreated().
```

7.4 Deterministic Tests — Eliminating Randomness

A deterministic test produces the same result every time it runs, in any environment, at any time. Non-determinism is the primary cause of flaky tests.

Common Non-Determinism Sources

- **DateTime.Now / DateTime.UtcNow:** The actual time changes. Solution: inject `IDateTimeProvider` interface, mock it in tests
- **Random / Guid.NewGuid():** Use fixed seed or inject `IRandomProvider`
- **External services:** Network calls fail intermittently. Always mock
- **Database shared state:** Order-dependent data. Use unique in-memory DB per test
- **Thread timing:** Thread.Sleep-based waits. Use Task-based async with proper awaiting

```
// Controlling DateTime in tests — production pattern
// Production interface
public interface IDateTimeProvider
{
    DateTime UtcNow { get; }
}
// Production implementation
public class SystemDateTimeProvider : IDateTimeProvider
{
    public DateTime UtcNow => DateTime.UtcNow;
}
// Service uses interface (injectable, testable)
public class BookingService
{
    private readonly IDateTimeProvider _dateTime;
    public BookingService(IDateTimeProvider dateTime) =>  dateTime = dateTime;
    public Booking CreateBooking()
    {
        return new Booking { CreatedAt =  dateTime.UtcNow }; // testable!
    }
}
// In test: inject fixed datetime
[Fact]
public void CreateBooking_SetsCreatedAtToCurrentTime()
{
    var fixedTime = new DateTime(2025, 1, 15, 10, 0, 0, DateTimeKind.Utc);
    var mockDateTime = new Mock<IDateTimeProvider>();
    mockDateTime.Setup(d => d.UtcNow).Returns(fixedTime);
    var service = new BookingService(mockDateTime.Object);
    var booking = service.CreateBooking();
    Assert.Equal(fixedTime, booking.CreatedAt); // deterministic!
}
```

7.5 Avoiding Flaky Tests — Production Discipline

Flaky tests are tests that pass and fail intermittently without code changes. They are **worse than no tests** — they erode trust in the entire test suite. Teams start ignoring failures, and real bugs slip through.

Flakiness Cause	Detection	Solution
Timing dependencies (sleep/delay)	Fails on slow CI machines	Use async/await properly; mock time
Shared database state	Passes alone, fails in suite	Unique DB per test; cleanup in teardown
Port conflicts	Fails when another process uses port	Use port 0 (OS-assigned) in TestServer
File system race conditions	Fails under parallel execution	Use unique temp paths per test
External service calls	Fails when service is down	Always mock external dependencies
Test order dependency	Fails when run in different order	Each test must be self-contained

Non-deterministic data	Random GUID comparisons	Fixed seed or mock random

7.6 Test Readability — Tests as Documentation

A test is read more often than it is written. Optimize for the reader. Key principles:

- Each test should test **one behavior** — if a test has 15 assertions, split it
- Use well-named variables: `expectedBookingId`, not `x`
- Avoid logic in tests (if statements, loops) — tests should be declarative, not imperative
- Use builder pattern for complex test data setup (prevents duplication)

```
// Builder pattern for test data
// Without builder (repeated, verbose, error-prone):
var booking = new BookingDto
{
    BookingId = 1, PassengerId = 5, SeatNumber = 'A1',
    DepartureDate = DateTime.Today, Status = 'Confirmed',
    TotalPrice = 150.00m, Currency = 'USD'
};
// With builder (clean, readable, DRY):
public class BookingDtoBuilder
{
    private int _bookingId = 1;
    private string  status = 'Confirmed';
    private string  seatNumber = 'A1';
    public BookingDtoBuilder WithBookingId(int id) {  bookingId = id; return this; }
    public BookingDtoBuilder WithStatus(string status) {  status = status; return this; }
    public BookingDtoBuilder WithSeatNumber(string seat) { _seatNumber = seat; return this;
 }
    public BookingDto Build() => new BookingDto
    {
        BookingId =  bookingId,
        Status =  status,
        SeatNumber =  seatNumber
        // Sensible defaults for everything else
    };
}
// Usage in tests — clean and explicit:
var booking = new BookingDtoBuilder().WithBookingId(42).WithStatus('Cancelled').Build();
```

SECTION 8

Advanced Testing Concepts — Integration, Auth, Middleware

8.1 Integration Testing with WebApplicationFactory — Deep Internals

WebApplicationFactory is the most powerful testing tool in the ASP.NET Core ecosystem. It creates a full in-memory application with real middleware, real DI, and real routing — without needing a real network.

```
// Advanced WebApplicationFactory with test overrides
public class CustomWebApplicationFactory : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.ConfigureServices(services =>
        {
            // 1. Replace real DbContext with in-memory
            RemoveService<DbContextOptions<AppDbContext>>(services);
            services.AddDbContext<AppDbContext>(opt =>
                opt.UseInMemoryDatabase($'IntegrationTest {Guid.NewGuid()}'));
            // 2. Replace real external services with fakes
            RemoveService<IEmailService>(services);
            services.AddSingleton<IEmailService, FakeEmailService>();
            // 3. Replace authentication with a test scheme
            services.AddAuthentication('Test')
                .AddScheme<AuthenticationSchemeOptions, TestAuthHandler>(
                    'Test', options => { });
        });
        builder.UseEnvironment('Testing');
    }
    private static void RemoveService<T>(IServiceCollection services)
    {
        var descriptor = services.SingleOrDefault(d => d.ServiceType == typeof(T));
        if (descriptor != null) services.Remove(descriptor);
    }
}
// Test class using the factory
public class BookingIntegrationTests : IClassFixture<CustomWebApplicationFactory>
{
    private readonly HttpClient  client;
    private readonly CustomWebApplicationFactory  factory;
    public BookingIntegrationTests(CustomWebApplicationFactory factory)
    {
         factory = factory;
        _client = factory.CreateClient();
```

```
    }
    [Fact]
    public async Task CreateAndRetrieveBooking_FullFlow_WorksCorrectly()
    {
        // Seed data using scoped service
        using var scope =  factory.Services.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        db.Passengers.Add(new Passenger { PassengerId = 1, Name = 'John Doe' });
        await db.SaveChangesAsync();
        // Create booking
        var createRequest = new { PassengerId = 1, SeatId = 5 };
        var createResponse = await  client.PostAsJsonAsync('/api/bookings', createRequest);
        Assert.Equal(HttpStatusCode.Created, createResponse.StatusCode);
        var location = createResponse.Headers.Location;
        Assert.NotNull(location);
        // Retrieve using Location header
        var getResponse = await  client.GetAsync(location);
        Assert.Equal(HttpStatusCode.OK, getResponse.StatusCode);
        var booking = await getResponse.Content.ReadFromJsonAsync<BookingDto>();
        Assert.Equal(1, booking.PassengerId);
    }
}
```

8.2 Authentication and Authorization Testing

Testing secured endpoints requires either configuring a test authentication handler or generating real JWT tokens. The test handler approach is simpler and more controllable.

```
// Test Authentication Handler — testing secured endpoints
// Custom auth handler that always authenticates the test user
public class TestAuthHandler : AuthenticationHandler<AuthenticationSchemeOptions>
{
    public TestAuthHandler(IOptionsMonitor<AuthenticationSchemeOptions> options,
        ILoggerFactory logger, UrlEncoder encoder) : base(options, logger, encoder) { }
    protected override Task<AuthenticateResult> HandleAuthenticateAsync()
    {
        // Create claims based on test header
        var claims = new[]
        {
            new Claim(ClaimTypes.Name, 'TestUser'),
            new Claim(ClaimTypes.Role, Request.Headers['X-Test-Role'].FirstOrDefault() ?? '
User'),
            new Claim('sub', '42'),
        };
        var identity = new ClaimsIdentity(claims, 'Test');
        var principal = new ClaimsPrincipal(identity);
        var ticket = new AuthenticationTicket(principal, 'Test');
        return Task.FromResult(AuthenticateResult.Success(ticket));
    }
```

```
}
// In tests — set role via header:
[Fact]
public async Task AdminEndpoint_AsAdmin_ReturnsOk()
{
    client.DefaultRequestHeaders.Add("X-Test-Role", "Admin");
    var response = await client.GetAsync("/api/admin/bookings");
    Assert.Equal(HttpStatusCode.OK, response.StatusCode);
}
[Fact]
public async Task AdminEndpoint_AsUser_ReturnsForbidden()
{
    client.DefaultRequestHeaders.Add("X-Test-Role", "User");
    var response = await client.GetAsync("/api/admin/bookings");
    Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
}
```

8.3 In-Memory Database Testing — Repository Tests

For testing repository implementations, EF Core's in-memory provider allows testing all CRUD operations without a real database. Important caveat: the in-memory provider does not enforce relational constraints — use SQLite in-memory mode for constraint testing.

```
// Repository unit test with in-memory EF Core
public class BookingRepositoryTests : IDisposable
{
    private readonly AppDbContext context;
    private readonly BookingRepository repository;
    public BookingRepositoryTests()
    {
        // Unique DB name ensures test isolation
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase($"BookingTest_{Guid.NewGuid()}")
            .Options;
        context = new AppDbContext(options);
        repository = new BookingRepository(context);
    }
    public void Dispose()
    {
        context.Database.EnsureDeleted(); // Clean up in-memory DB
        context.Dispose();
    }
    [Fact]
    public async Task GetByIdAsync_WhenBookingExists_ReturnsBooking()
    {
        // Arrange: directly add to in-memory DB (no mock needed!)
        var booking = new Booking { BookingId = 1, PassengerId = 5 };
        context.Bookings.Add(booking);
        await context.SaveChangesAsync();
```

```
        // Act
        var result = await  repository.GetByIdAsync(1);
        // Assert
        Assert.NotNull(result);
        Assert.Equal(5, result.PassengerId);
    }
    [Fact]
    public async Task AddAsync SavesBookingCorrectly()
    {
        // Arrange
        var newBooking = new Booking { PassengerId = 10, SeatId = 3 };
        // Act
        await  repository.AddAsync(newBooking);
        // Assert: verify it's in the database
        var saved = await  context.Bookings.FindAsync(newBooking.BookingId);
        Assert.NotNull(saved);
        Assert.Equal(10, saved.PassengerId);
    }
}
```

8.4 Middleware Testing

Custom middleware can be tested in isolation using ASP.NET Core's `TestServer` or by unit testing the `InvokeAsync` method directly.

```
// Testing custom middleware directly
// Custom middleware example
public class RequestIdMiddleware
{
    private readonly RequestDelegate  next;
    public RequestIdMiddleware(RequestDelegate next) =>  next = next;
    public async Task InvokeAsync(HttpContext context)
    {
        context.Response.Headers['X-Request-Id'] = Guid.NewGuid().ToString();
        await  next(context);
    }
}
// Unit test using DefaultHttpContext
[Fact]
public async Task RequestIdMiddleware AddsRequestIdHeader()
{
    // Arrange
    var context = new DefaultHttpContext();
    var nextCalled = false;
    RequestDelegate next = (ctx) => { nextCalled = true; return Task.CompletedTask; };
    var middleware = new RequestIdMiddleware(next);
    // Act
    await middleware.InvokeAsync(context);
    // Assert
```

```
    Assert.True(nextCalled); // middleware called next
    Assert.True(context.Response.Headers.ContainsKey('X-Request-Id'));
    Assert.True(Guid.TryParse(context.Response.Headers['X-Request-Id'], out  ));
}
```

8.5 Exception Testing — Correct Patterns

```
// All exception testing patterns
// Pattern 1: Assert.Throws<T> for synchronous code
[Fact]
public void Service WithNullInput ThrowsArgumentNullException()
{
    var ex = Assert.Throws<ArgumentNullException>(
        () =>  service.Process(null));
    Assert.Equal('request', ex.ParamName);  // verify correct parameter
}
// Pattern 2: Assert.ThrowsAsync<T> for async code
[Fact]
public async Task Service_WithInvalidId_ThrowsKeyNotFoundException()
{
    var ex = await Assert.ThrowsAsync<KeyNotFoundException>(
        () => _service.GetByIdAsync(-1));
    Assert.Contains('not found', ex.Message);
}
// Pattern 3: Verify controller handles exception with correct HTTP status
[Fact]
public async Task Controller WhenServiceThrows Returns500()
{
     mockService.Setup(s => s.GetByIdAsync(1))
        .ThrowsAsync(new InvalidOperationException('DB unavailable'));
    var result = await  controller.GetBooking(1);
    Assert.IsType<ObjectResult>(result);
    Assert.Equal(500, ((ObjectResult)result).StatusCode);
}
// WRONG: this test always passes because the exception is never captured
// [Fact]
// public async Task Wrong ExceptionTest()
// {
//     try { await  service.GetByIdAsync(-1); }
//     catch (Exception) { /* swallowed */ }
//     Assert.True(true); // meaningless — test always passes
// }
```

8.6 Cancellation Token Testing

In production, cancellation tokens propagate request cancellation through the call stack. Testing that your code respects cancellation is important for preventing hung requests.

```
// Cancellation token testing
```

```
[Fact]
public async Task GetBooking_WhenCancelled_ThrowsOperationCancelled()
{
    // Arrange: create a pre-cancelled token
    var cts = new CancellationTokenSource();
    cts.Cancel(); // already cancelled

     mockService
        .Setup(s => s.GetByIdAsync(1, It.IsAny<CancellationToken>()))
        .ThrowsAsync(new OperationCanceledException());

    // Act & Assert
    await Assert.ThrowsAsync<OperationCanceledException>(
        () =>  controller.GetBooking(1, cts.Token));
}
// Also verify token is passed through:
[Fact]
public async Task GetBooking_PassesCancellationTokenToService()
{
    var cts = new CancellationTokenSource();
    CancellationToken capturedToken = default;

     mockService
        .Setup(s => s.GetByIdAsync(1, It.IsAny<CancellationToken>()))
        .Callback<int, CancellationToken>((id, ct) => capturedToken = ct)
        .ReturnsAsync(new BookingDto());

    await  controller.GetBooking(1, cts.Token);

    Assert.Equal(cts.Token, capturedToken); // token was passed through
}
```

SECTION 9

Debugging & Interview / Viva Preparation

9.1 The Master Viva Question Bank

The following questions are organized from foundational to advanced. Each is designed to probe real understanding, not memorized answers.

Category 1: Conceptual Foundations

Q: What is the difference between a mock, stub, fake, spy, and dummy?

A: Dummy: passed but unused (just satisfies parameter requirements). Stub: configured to return specific values; no interaction verification. Fake: simplified real implementation (in-memory repo). Spy: records invocations for manual inspection; looser than mocks. Mock: pre-configured with expectations; verifies interactions via Verify(). In Moq, Mock can function as stub (Setup only) or mock (Setup + Verify).

Q: Why should tests be independent of each other?

A: Because dependent tests are non-deterministic: if TestA fails, TestB fails for wrong reasons. CI pipelines run tests in parallel and in arbitrary order. A test that relies on state from another test will produce different results depending on execution order — a definition of a flaky test. Independence is enforced by: (1) new instance per test (xUnit), (2) unique in-memory DB per test, (3) no static mutable state.

Q: Why are interfaces preferred over concrete classes for testing?

A: Interfaces create seams — points where real implementations can be replaced with fakes. Without an interface, a class is a closed box — you cannot replace its behavior without modifying its source. With an interface, Moq can generate a proxy implementation. This is the Dependency Inversion Principle (D in SOLID): depend on abstractions, not concretions. Code written against interfaces is inherently more testable, more flexible, and more maintainable.

Q: What happens if you don't call Verify() in a Moq test?

A: The test only validates the return value behavior (Assert), not the interaction behavior. If a bug causes the wrong service method to be called (or not called at all), but the test's assertions still pass (because the return value was mocked correctly), the bug is undetected. Omitting Verify() means you're doing state testing but not interaction testing. Always Verify() when the SERVICE CALL ITSELF is part of the expected behavior.

Category 2: Internal Mechanisms

Q: What is Castle.DynamicProxy and how does Moq use it?

A: Castle.DynamicProxy is a library that generates real .NET classes at runtime using Reflection.Emit (IL code generation). Moq embeds it. When you call new Mock(), DynamicProxy generates a new class (in a dynamic assembly) that implements IService. Every method in this generated class calls Moq's Interceptor. The interceptor checks against configured setups and returns the configured value. The dynamically generated class is a real CLR type — it can be assigned to IService variables.

Q: How does Assert.Equal work for complex objects?

A: Assert.Equal uses EqualityComparer.Default, which uses: (1) IEquatable.Equals() if implemented, (2) object.Equals() overrides, (3) reference equality as fallback. For custom DTOs without IEquatable, two different instances with identical fields are NOT equal by default — you must override Equals() and GetHashCode(), or use Assert.Equivalent() (xUnit 2.4.2+) which does deep property comparison, or use FluentAssertions.

Q: How does xUnit ensure test isolation at the memory level?

A: xUnit creates a new instance of the test class for every [Fact] method. This means: (1) All instance fields declared in the constructor are re-initialized per test — no shared state. (2) Mocks are fresh — no invocation records from previous tests. (3) The controller is a fresh instance with fresh mock injected. After each test, xUnit calls Dispose() if the class implements IDisposable, then abandons the instance for GC collection. The next test gets a completely new object graph.

Q: What is MethodInfo.Invoke() and why does xUnit use it?

A: MethodInfo.Invoke() is the reflection API for calling a method by its metadata representation, rather than directly. xUnit uses it because: (1) the test runner doesn't know test method signatures at compile time — they're discovered at runtime via reflection, (2) it allows wrapping in try-catch to capture any thrown exception, (3) it supports passing discovered parameters (for [Theory] tests). The cost is a small reflection overhead (~100ns), negligible compared to test logic.

Category 3: Advanced Scenarios

Q: When would you choose integration testing over unit testing?

A: Choose integration testing when: (1) testing that DI registrations are correct (types resolve), (2) testing that middleware executes in the correct order, (3) testing that routing maps to correct controllers, (4) testing that serialization/deserialization works correctly end-to-end, (5) testing that EF Core queries produce correct SQL, (6) testing authentication and authorization policies as configured. Unit tests cannot catch configuration errors — only integration tests can.

Q: How would you test a controller method that uses [Authorize] in a unit test?

A: The [Authorize] attribute is enforced by ASP.NET Core's authorization middleware, which does NOT run during unit tests (you call the controller method directly). In unit tests, [Authorize] is essentially invisible. To test authorization behavior: (1) use WebApplicationFactory for integration tests, (2) inject IAuthorizationService as a dependency and mock it in unit tests, (3) test that the controller calls authorizationService.AuthorizeAsync() correctly. For full auth pipeline testing, integration tests with TestAuthHandler are required.

Q: What is the difference between Moq's Loose and Strict behavior, and when does it matter?

A: Loose (default): unconfigured method calls return default(T) — null for reference types, 0 for int. No exception. Strict: unconfigured calls immediately throw MockException. Use Strict when: you want to be explicit about every dependency interaction, catch accidental calls during refactoring. Strict makes tests more brittle (adding a new logging call breaks strict tests) but more precise. Use Loose for integration-style unit tests; Strict for pure unit tests where every interaction matters.

Q: Your test suite has 200 tests and takes 15 minutes to run. How do you diagnose and fix this?

A: Step 1: Profile which tests are slow (dotnet test --logger 'trx;verbosity=normal' + analyze TRX). Step 2: Check if slow tests are hitting real databases, external services, or using Thread.Sleep. Step 3: Check for unnecessary sequential execution (ICollectionFixture overuse). Step 4: Enable parallel test execution in xunit.runner.json. Step 5: Replace slow integration tests with unit tests where possible. Step 6: Move slow integration tests to a separate 'slow' test project run less frequently. Target: unit tests under 5ms, integration tests under 1s, E2E under 30s.

9.2 Interview Trap Questions — What They're Really Testing

These questions seem simple but probe deep understanding:

Trap Question	Shallow Answer (Wrong)	Deep Answer (Right)
'Does mocking make your tests slower?'	'No, mocks are fast'	'No — mocks eliminate I/O. Each mock call is a pure in-memory operation via t

'Can you mock a private method?'	'No'	'Private methods are implementation details — you should not test them directl
'When should you not write unit tests?'	'Always write unit tests'	'Diminishing returns appear for: pure I/O code (CRUD with no business logic), f
'Is 100% code coverage the goal?'	'Yes, 100% is best'	'100% is a vanity metric. You can have 100% coverage with meaningless tests

9.3 Production Debugging Cases — Real-World Scenarios

Scenario 1: Test Passes Locally, Fails in CI

Root causes: (1) Different datetime/timezone (use IDateTimeProvider), (2) Test order dependency (CI runs tests in different order), (3) Missing environment variables in CI, (4) File path differences (Unix vs Windows), (5) Parallel execution causing shared state corruption.

Debug approach: Run locally with same environment variables. Run with `dotnet test --no-parallel` to eliminate parallelism. Add logging to capture state at failure point.

Scenario 2: Mock Returns Null Even After Setup()

Root causes: (1) Setup uses concrete argument but Actual call uses different value — argument mismatch. (2) Non-virtual method on concrete class — setup is silently ignored. (3) Setup called on wrong mock instance. (4) Async method not using ReturnsAsync().

Debug: Print `mock.Invocations` after the call to see what arguments were actually passed. Switch to `It.IsAny<T>()` to confirm setup works, then narrow to specific matcher.

Scenario 3: Assert.Equal Fails for Objects That Look Identical

Root cause: Reference equality fallback. Two `new BookingDto { Id = 1 }` are different references. Solution: (1) Override Equals()/GetHashCode() in DTO, (2) Use Assert.Equivalent() (xUnit 2.4.2+), (3) Compare individual properties, (4) Use FluentAssertions: `result.Should().BeEquivalentTo(expected)`.

SECTION 10

Enterprise Testing Architecture — CI/CD, Coverage, and Scale

10.1 Enterprise Project Test Structure

In production enterprise .NET projects, tests are organized into separate projects with clear responsibility boundaries. Never mix test types in one project.

```
// Enterprise solution structure
BusTicketBooking.sln
■■■ src/
■   ■■■ BusTicketBooking.API/          # ASP.NET Core Web API
■   ■■■ BusTicketBooking.Application/  # Business logic, services, DTOs
■   ■■■ BusTicketBooking.Domain/       # Entities, domain events, exceptions
■   ■■■ BusTicketBooking.Infrastructure/ # EF Core, external services
■
■■■ tests/
    ■■■ BusTicketBooking.UnitTests/
    ■   ■■■ Controllers/              # Controller tests (mocked services)
    ■   ■■■ Services/                 # Service tests (mocked repos)
    ■   ■■■ Domain/                   # Domain logic tests (no mocks)
    ■   ■■■ Builders/                 # Test data builders
    ■
    ■■■ BusTicketBooking.IntegrationTests/
    ■   ■■■ Api/                      # WebApplicationFactory tests
    ■   ■■■ Repositories/             # Real DB (in-memory or TestContainers)
    ■   ■■■ Infrastructure/           # External service integration
    ■
    ■■■ BusTicketBooking.E2ETests/     # Playwright / Postman collection tests
# Each test project has its own .csproj with appropriate NuGet references:
# UnitTests: xunit, Moq, Microsoft.NET.Test.Sdk, coverlet.collector
# IntegrationTests: above + Microsoft.AspNetCore.Mvc.Testing, EF InMemory
# E2ETests: above + Microsoft.Playwright or RestSharp
```

10.2 CI/CD Pipeline Integration — Azure DevOps / GitHub Actions

Tests run automatically on every commit in a properly configured pipeline. The pipeline gates — no deployment proceeds if tests fail.

```
// GitHub Actions test pipeline (.github/workflows/ci.yml)
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
```

```
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET 8
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'
      - name: Restore dependencies
        run: dotnet restore
      - name: Build
        run: dotnet build --no-restore --configuration Release
      - name: Run Unit Tests with Coverage
        run: |
          dotnet test tests/BusTicketBooking.UnitTests/ \
            --no-build --configuration Release \
            --collect:'XPlat Code Coverage' \
            --results-directory ./coverage \
            --logger 'trx;LogFileName=unit-results.trx'
      - name: Run Integration Tests
        run: |
          dotnet test tests/BusTicketBooking.IntegrationTests/ \
            --no-build --configuration Release \
            --logger 'trx;LogFileName=integration-results.trx'
      - name: Generate Coverage Report
        run: |
          dotnet tool install -g dotnet-reportgenerator-globaltool
          reportgenerator \
            -reports:./coverage/**/coverage.cobertura.xml \
            -targetdir:coveragereport \
            -reporttypes:Html;Cobertura
      - name: Publish Test Results
        uses: EnricoMi/publish-unit-test-result-action@v2
        if: always()
        with:
          files: '**/*.trx'
      - name: Publish Coverage Report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coveragereport/
      - name: Check Coverage Threshold
        run: |
          dotnet test tests/BusTicketBooking.UnitTests/ \
            /p:CollectCoverage=true \
            /p:CoverletOutput=coverage/ \
```

```
        /p:CoverageFailUnder=80  # Fails pipeline if below 80%
```

10.3 SonarQube Integration — Code Quality Gating

SonarQube/SonarCloud provides static analysis alongside test results. Key metrics it tracks:

SonarQube Metric	What It Measures	Typical Gate
Coverage	% of code executed by tests	>= 80%
Duplications	% of duplicated code blocks	<= 3%
Maintainability Rating	Technical debt ratio	A rating
Reliability Rating	Bug density	A rating
Security Rating	Vulnerability count	A rating
Cognitive Complexity	Code readability/complexity	<= 15 per method
Test Success Rate	% of tests passing	100%

10.4 Testing in Microservices Architecture

Microservices testing introduces unique challenges: service-to-service communication, distributed state, and eventual consistency. The testing strategy must address all layers.

Contract Testing — Pact

When Service A calls Service B's API, both must agree on the contract (request/response format). **Consumer-Driven Contract Testing** (CDCT) with Pact allows each service to test independently while ensuring contracts are honored.

- Consumer (Service A) defines expected contract → generates Pact file
- Provider (Service B) verifies it can satisfy the contract → runs Pact verification
- Pact broker stores and distributes contract files
- Both sides can deploy independently if contracts are verified

Layered Testing Strategy for Microservices

Layer	Test Type	Tool	Frequency
Service internals	Unit tests	xUnit + Moq	Every commit
Service API	Integration tests	WebApplicationFactory	Every commit
Service contracts	Contract tests	PactNet	Every commit
Service cluster	Component tests	TestContainers	PR / Daily
Full system	E2E tests	Playwright / k6	Daily / Release
Performance	Load tests	k6 / Azure Load Testing	Weekly / Release

10.5 TestContainers — Real Dependencies in CI

TestContainers for .NET allows spinning up real Docker containers (SQL Server, Redis, RabbitMQ) as part of tests. This gives the confidence of real integration without the flakiness of shared infrastructure.

```
// TestContainers with SQL Server
// Install: Testcontainers.MsSql NuGet package
public class DatabaseIntegrationTests : IAsyncLifetime
{
    private readonly MsSqlContainer  sqlContainer;
    private AppDbContext  context;
    public DatabaseIntegrationTests()
    {
        // Defines the Docker container to spin up
         sqlContainer = new MsSqlBuilder()
            .WithImage('mcr.microsoft.com/mssql/server:2022-latest')
            .WithPassword('YourStr0ngPassw@rd')
            .Build();
    }
    // IAsyncLifetime: runs before first test in class
    public async Task InitializeAsync()
    {
        await  sqlContainer.StartAsync();  // starts Docker container
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseSqlServer( sqlContainer.GetConnectionString())
            .Options;
         context = new AppDbContext(options);
        await  context.Database.MigrateAsync(); // apply real migrations
    }
    // IAsyncLifetime: runs after all tests in class
    public async Task DisposeAsync()
    {
        await  context.DisposeAsync();
        await  sqlContainer.DisposeAsync(); // stops and removes container
    }
    [Fact]
    public async Task CreateBooking WithRealSqlServer PersistsCorrectly()
    {
        var repo = new BookingRepository( context);
        var booking = new Booking { PassengerId = 1, SeatId = 5 };
        await repo.AddAsync(booking);
        var saved = await repo.GetByIdAsync(booking.BookingId);
        Assert.NotNull(saved);
        Assert.Equal(1, saved.PassengerId);
    }
}
```

10.6 Mutation Testing — Verifying Your Test Quality

Mutation testing is the ultimate test quality metric. It automatically modifies your production code in small ways (mutations) — changing > to >=, removing a return statement, negating a condition — and checks if

your tests catch these changes (kill the mutants).

- **Mutation score:** % of mutations caught by tests. High score = high-quality tests
- **Survived mutants:** code changes that tests didn't detect = gaps in test coverage
- **Tool:** Stryker.NET — `dotnet stryker`

A high code coverage score does not mean your tests are meaningful. A 95% coverage test suite that never asserts anything meaningful will have a 0% mutation score. This is why mutation testing is the gold standard for test quality measurement.

10.7 Final Viva Questions — Enterprise Level

Q: How do you decide what percentage of tests should be unit vs integration?

A: Start with the Test Pyramid principle: 70-80% unit, 15-25% integration, 5-10% E2E. But adjust based on your architecture. If you have thick services with complex business logic: more unit tests. If you have thin services that primarily orchestrate external calls: more integration tests. The goal is maximum confidence at minimum cost. Always ask: 'What is the cheapest test that can catch this bug?'

Q: What is consumer-driven contract testing and why does it matter in microservices?

A: Consumer-driven contract testing (CDCT) solves the microservice integration problem: how do you test that Service A and Service B still communicate correctly without deploying both simultaneously? The consumer (A) defines a Pact contract of expected API behavior. The provider (B) runs the Pact verifier against this contract in its own CI. If B breaks the contract, B's CI fails before deployment. Both services can be tested and deployed independently.

Q: How do you handle a failing test in production CI?

A: Step 1: Never skip the test as a quick fix — it hides a problem. Step 2: Reproduce locally. Step 3: Classify: is it a flaky test or a real bug? Flaky: fix the underlying non-determinism (shared state, timing, external dependency). Real bug: the code is broken — fix the code, not the test. If the test reveals a legitimate behavior change (requirements changed), update both code and test, with a clear commit message explaining the requirement change.

APPENDIX — Quick Reference Card

xUnit Cheat Sheet

xUnit Feature	Syntax / Code
Basic test	[Fact] public void Test() { }
Async test	[Fact] public async Task Test() { }
Parameterized	[Theory] [InlineData(1,true)] public void T(int x, bool y)
Member data	[Theory] [MemberData(nameof(Cases))] public void T(...)
Skip test	[Fact(Skip = 'reason')]
Shared fixture	class Tests : IClassFixture<MyFixture>
Test output	constructor: ITestOutputHelper output

Collection	[Collection('Name')] on test class
Disable parallel	[assembly: CollectionBehavior(DisableTestParallelization=true)]
Timeout	[Fact(Timeout = 5000)] (milliseconds)

Moq Cheat Sheet

Moq Feature	Syntax
Create mock	var mock = new Mock<IService>()
Get object	mock.Object
Setup return	mock.Setup(s => s.Method(1)).Returns(value)
Async return	mock.Setup(s => s.Method()).ReturnsAsync(value)
Throw exception	mock.Setup(s => s.Method()).Throws<Exception>()
Async throw	mock.Setup(s => s.Method()).ThrowsAsync(new Ex())
Any argument	It.IsAny<int>()
Conditional arg	It.Is<int>(x => x > 0)
Callback	.Callback<int>(id => captured = id)
Sequential	mock.SetupSequence(s => s.M()).Returns(1).Returns(2)
Verify once	mock.Verify(s => s.Method(1), Times.Once())
Verify never	mock.Verify(s => s.Method(It.IsAny<int>()), Times.Never())
Verify all	mock.VerifyAll()
Strict mode	new Mock<IService>(MockBehavior.Strict)
Property mock	mock.SetupProperty(s => s.IsActive, true)

Assert Cheat Sheet

Assertion	Use Case
Assert.Equal(expected, actual)	Value equality
Assert.NotEqual(a, b)	Values are different
Assert.True(condition)	Boolean is true
Assert.False(condition)	Boolean is false
Assert.Null(obj)	Object is null
Assert.NotNull(obj)	Object is not null
Assert.IsType<T>(obj)	Exact runtime type check
Assert.IsAssignableFrom<T>(obj)	Type compatibility check
Assert.Throws<T>(() => action())	Sync exception verification

Assert.ThrowsAsync<T>(() => task)	Async exception verification
Assert.Empty(collection)	Collection has zero elements
Assert.Single(collection)	Collection has exactly one element
Assert.Contains(item, collection)	Item exists in collection
Assert.DoesNotContain(item, col)	Item not in collection
Assert.Collection(list, elem1, ...)	Ordered element inspection
Assert.Equivalent(expected, actual)	Deep property equality (xUnit 2.4.2+)
Assert.Multiple(() => { ... })	Run multiple asserts, report all failures

---

END OF HANDBOOK

ASP.NET Core Testing Handbook — xUnit + Moq + Deep Internals

This handbook covered: Testing Fundamentals → Types of Testing → xUnit Internals → Moq Internals → Controller Testing → Complete Execution Flow → Best Practices → Advanced Concepts → Debugging → Enterprise Architecture. Apply these concepts progressively: start with solid unit tests for business logic, add integration tests for critical paths, and build toward a comprehensive automated test suite that gives your team deployment confidence.
